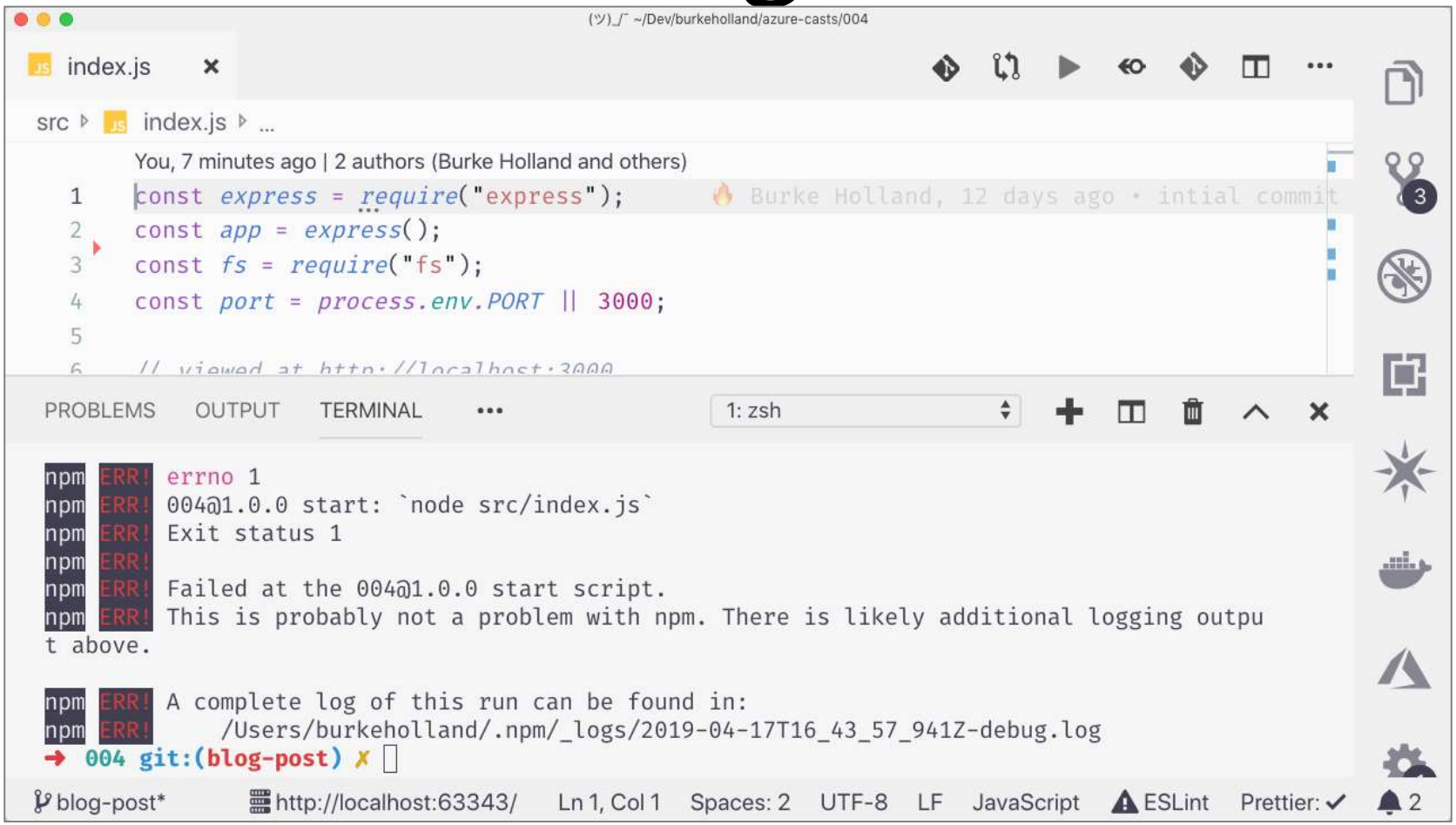
Tehnologii Web

avansat



programare Web: Node.js – supliment

Dr. Sabin Corneliu Buraga – profs.info.uaic.ro/~busaco/

„Cine a văzut vreodată o bijuterie frumos cizelată de bijutier cu ajutorul ciocanului?”

Jan Amos Comenius

JavaScript (ECMAScript)

un limbaj de programare pentru Web

Inventat de Brendan Eich (1995)

detalii în cadrul prelegerilor

Limbajul de programare JavaScript.
Aspecte esențiale

profs.info.uaic.ro/~busaco/teach/courses/staw/web-film.html#week2

Limbajul de programare JavaScript.
Aspecte moderne: ES6 et al.

profs.info.uaic.ro/~busaco/teach/courses/staw/web-film.html#week3

a se considera și

Ingineria dezvoltării aplicațiilor JavaScript

profs.info.uaic.ro/~busaco/teach/courses/staw/web-film.html#week7

Cum dezvoltăm cu JavaScript aplicații Web la nivel de server?

node.js: caracterizare

*"**Node.js**® is a JavaScript runtime built on Chrome's V8 JavaScript engine."*

"Node.js *uses an* event-driven, non-blocking I/O *model that makes it lightweight and efficient."*

node.js: caracterizare

"Node.js *is designed to build* scalable network applications."

"Node.js *is an* open-source, cross-platform, JavaScript runtime environment *that executes JavaScript code outside of a browser.*"

"Node.js *represents a* JavaScript everywhere *paradigm.*"

node.js: caracterizare

Oferă suport pentru cele mai importante protocoale Web și Internet

HTTP (*HyperText Transfer Protocol*)
DNS (*Domain Name System*)
TLS (*Transport Layer Security*)
functionalități de nivel scăzut (*socket*-uri TCP)

Ryan Dahl concepe **Node.js** (2009) – **nodejs.org**

rulează pe mașini respectând POSIX + Windows (2011)

adoptat de industrie (din 2012)

e.g., Cloud9 IDE, eBay, Google, LinkedIn, NASA, Netflix, Walmart

hackernoon.com/@nodejs

Node.js Foundation (2015)

IBM, Intel, Joyent, Microsoft, RedHat, SAP, The Linux Foundation,…

Node.js 14.16 LTS – *Long Term Support* (23 feb. 2021)

versiune stabilă actuală pentru uz în producție

Node.js 15.11 (3 martie 2021)

versiunea cea mai recentă

github.com/nodejs/node

node.js

Oferă un mediu de execuție în linia de comandă, pe baza unor biblioteci C++ și a procesorului V8

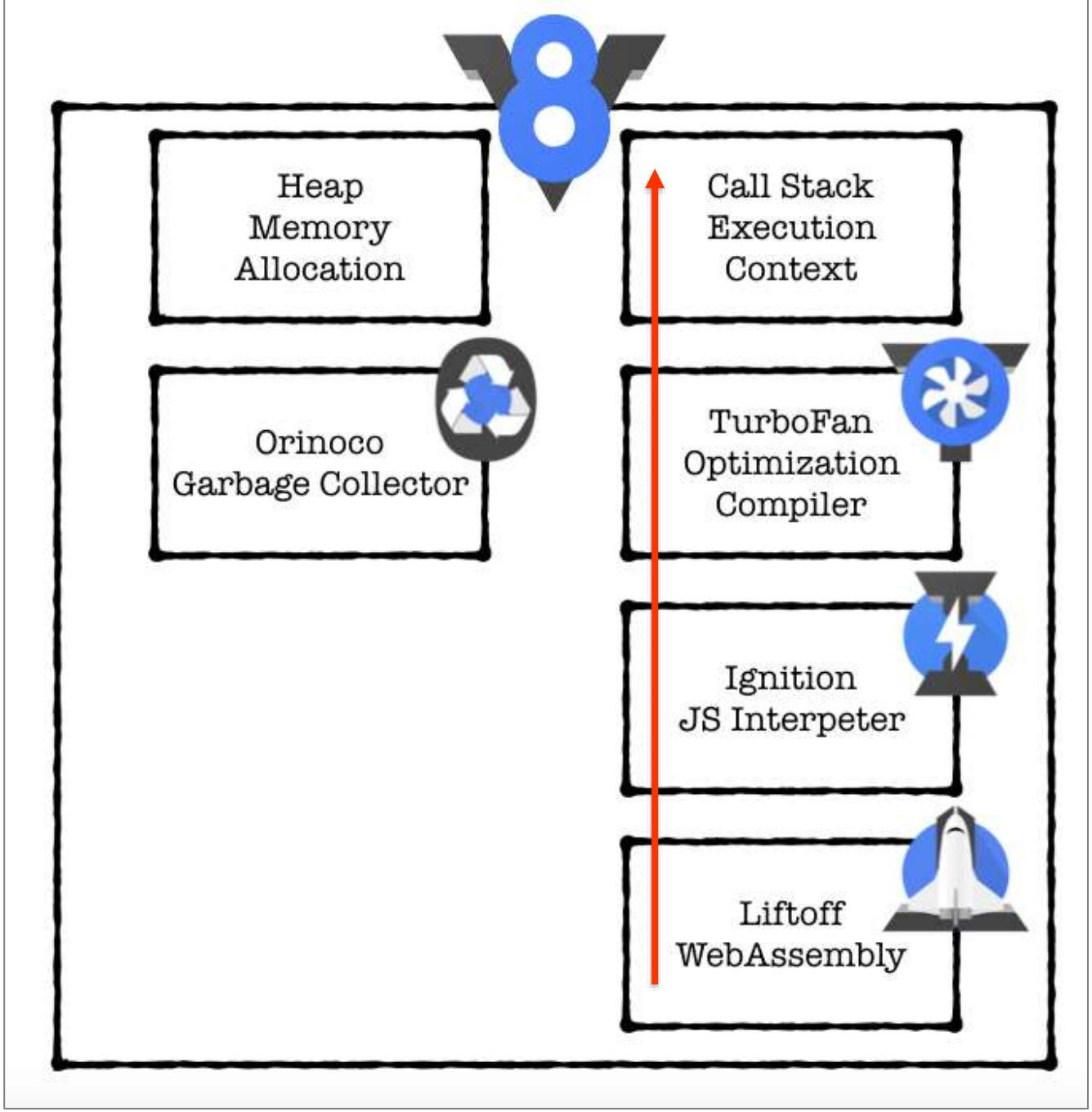
node program.js

node.js

Permite dezvoltarea de **aplicații Web**
la nivel de server în limbajul JavaScript

recurge la V8
procesor – compilează + execută codul – ECMAScript
a single threaded execution JS engine
creat de Google, implementat în C++ și JS, disponibil liber

v8.dev

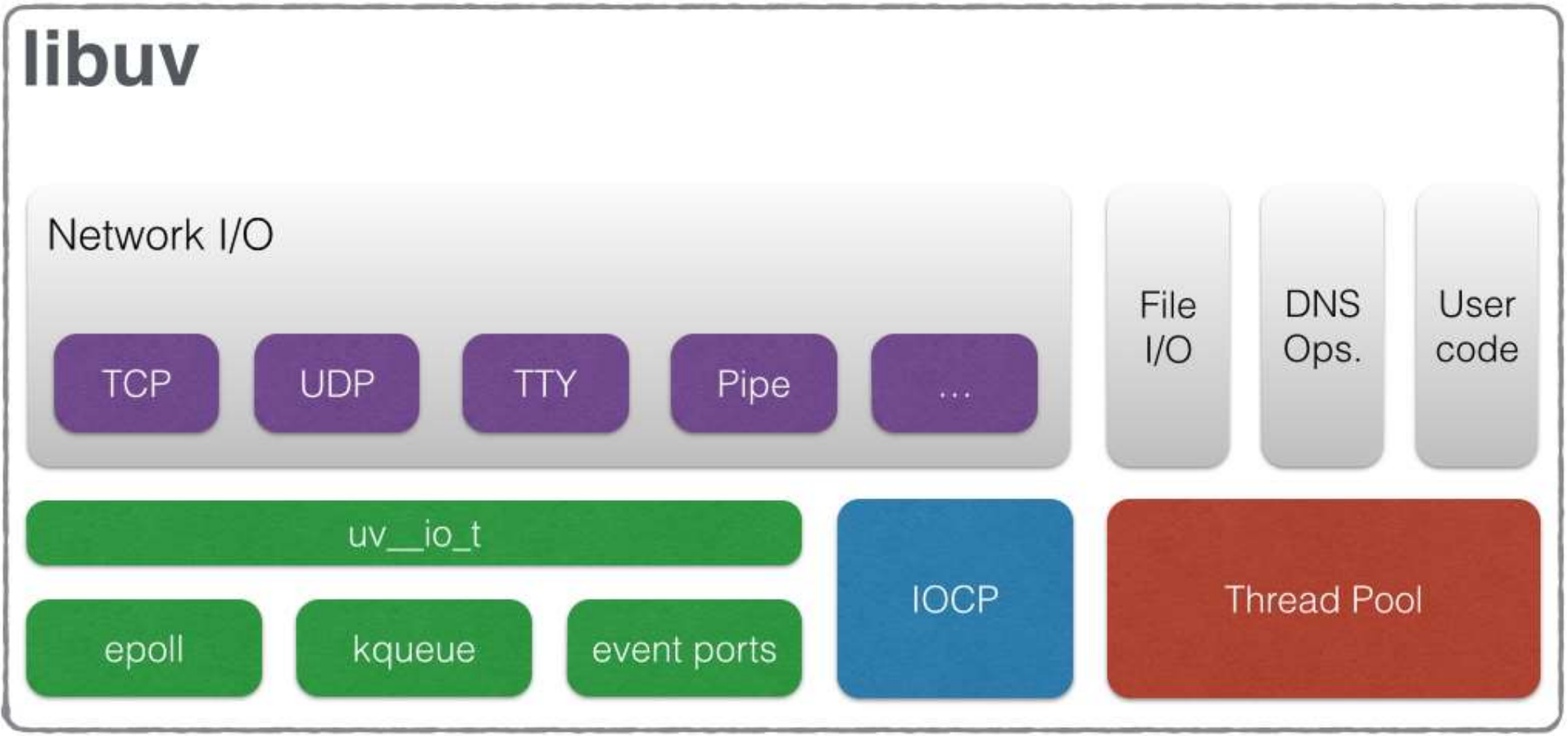


v8.dev/blog/ignition-interpreter

node.js

Permite dezvoltarea de **aplicații Web**
la nivel de server în limbajul JavaScript

utilizează libluv
bibliotecă multi-platformă disponibilă liber
axată pe efectuarea de operații I/O asincrone
implementare C++
libuv.org



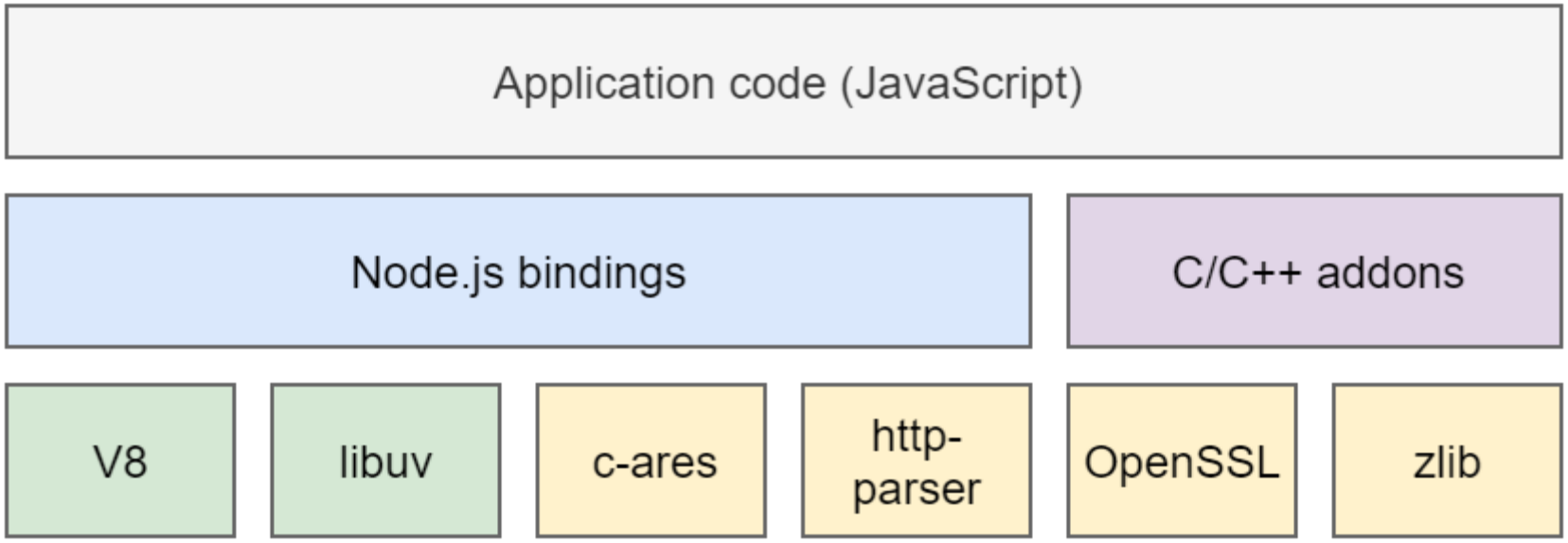
*"libuv uses a thread pool to make asynchronous file I/O operations possible, but network I/O is **always** performed in a single thread, each loop's thread"*

docs.libuv.org/en/v1.x/design.html

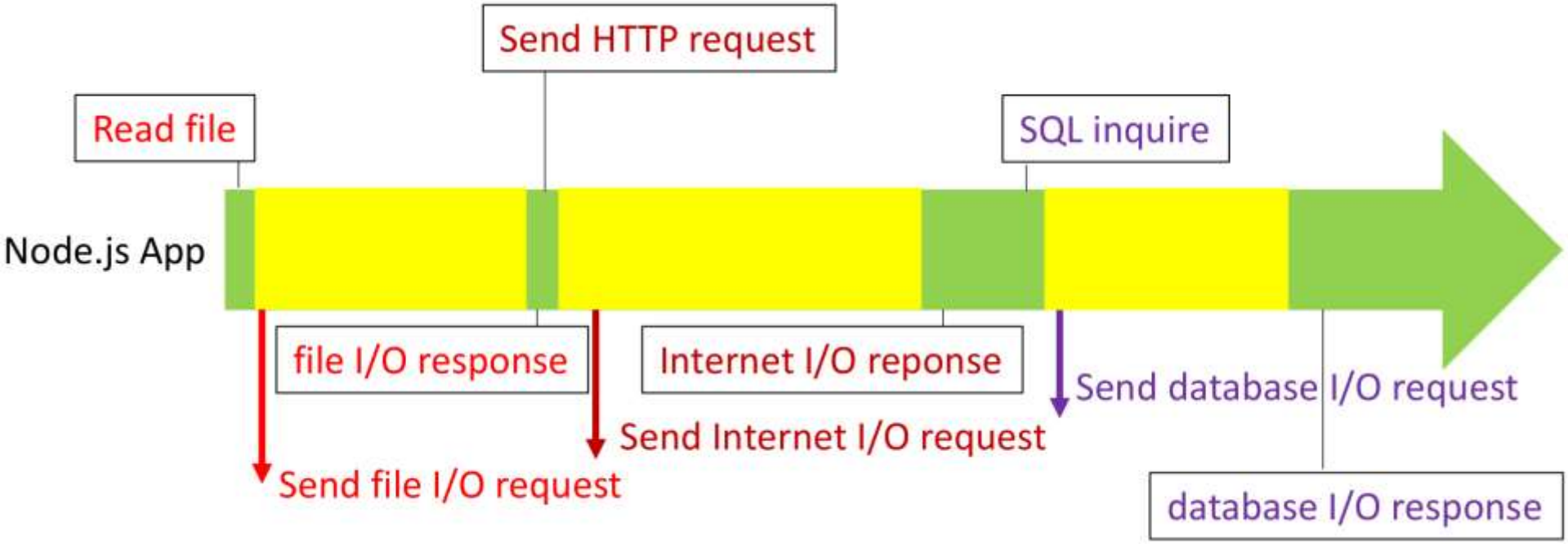
node.js: caracterizare

Intern, mediul recurge la
diverse biblioteci *open source* C sau C++

libuv (*asynchronous I/O*) – **docs.libuv.org**
c-ares (*asynchronous DNS requests*) – **c-ares.haxx.se**
OpenSSL (*crypto* + *SSL/TLS*) – **www.openssl.org**
zlib (*compression*) – **zlib.net**
V8 (*JavaScript engine*) – **v8docs.nodesource.com**


Application code (JavaScript)
Node.js bindings
C/C++ addons
V8
libuv
c-ares
http-parser
OpenSSL
zlib

Uzual, o aplicație Web realizează un număr mare de operații – în acest caz, asincrone – de intrare/ieșire



G. Yeh (2014): www.slideshare.net/garyyeh165/basic-understanding-and-implement-of-nodejs

node.js: caracterizare

Operațiile de intrare/ieșire sunt **asincrone**

fiecare cerere (operație) adresată aplicației – *e.g.*, acces la disc, la rețea, la alt proces – poate avea atașată o funcție de tratare a unui eveniment specific

evented I/O

cod JS executat de client (*browser* Web)		cod JS rulat pe partea de server (node.js)
așteaptă și tratează evenimente de interacțiune (onclick, onmouseover, onkeypressed,…)	**procesare bazată pe evenimente** *evented/ event-based*	așteaptă și tratează cereri (evenimente) provenite de la client(i)
programul trebuie să fie responsiv atunci când așteaptă încărcarea datelor de pe rețea (*e.g.*, JSON, XML, imagini, video) via Ajax/Comet ori *Web sockets*	**asincronism** (*e.g.*, operatii neblocante)	programul trebuie să fie responsiv atunci când așteaptă încărcarea datelor locale/externe (preluate din baze de date, fișiere, servicii Web, API-uri,…)

adaptare după Ben Sheldon (2012)

node.js: caracterizare

O aplicație node.js rulează într-**un singur proces**

event loop ≡ "an entity that handles & processes external ***events*** *and converts them into* ***callback*** *invocations"*

blog.mixu.net/2011/02/01/understanding-the-node-js-event-loop/

node.js: caracterizare

O aplicație node.js rulează într-**un singur proces**

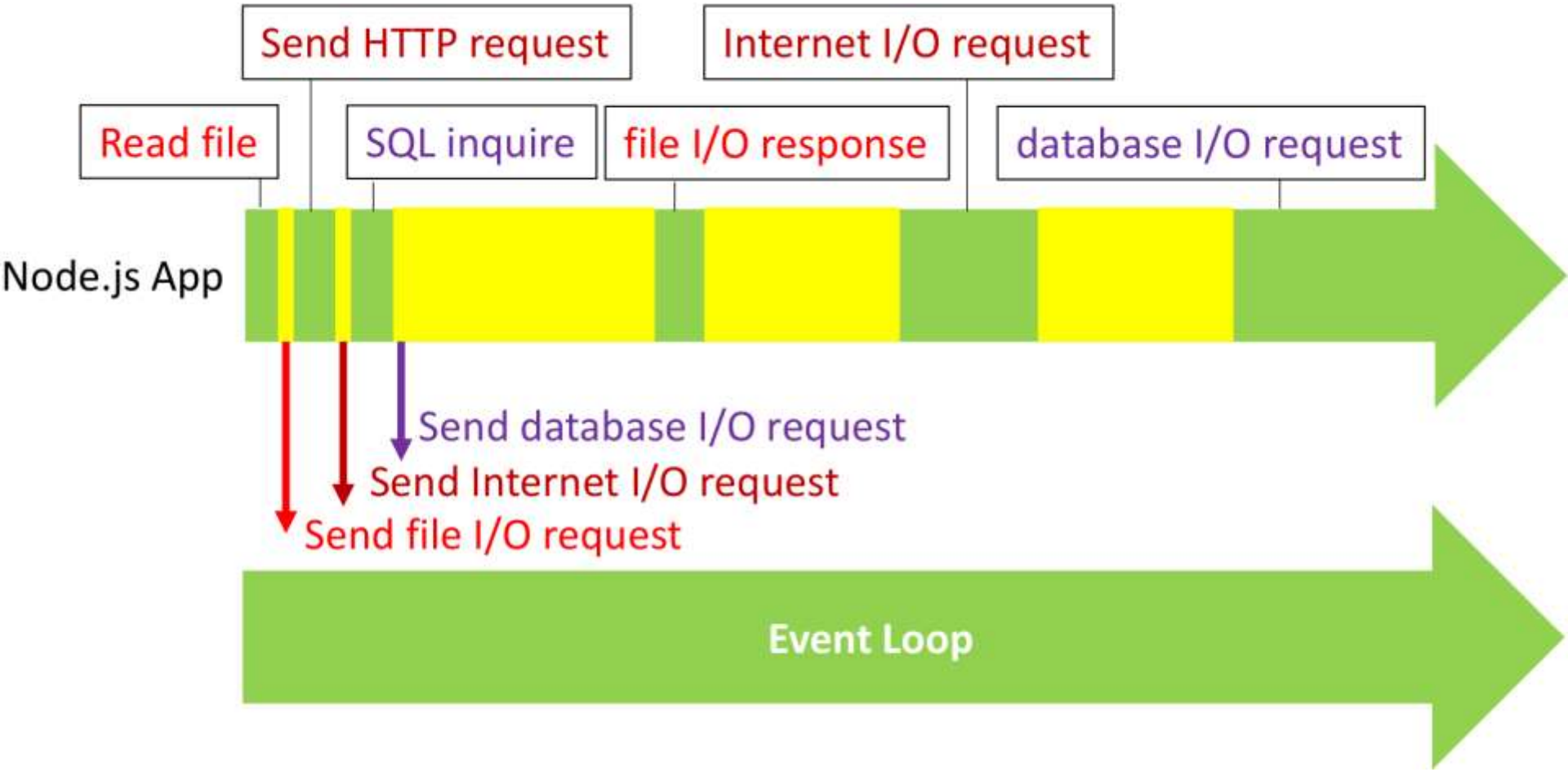
event loop ≡ "an entity that handles & processes external ***events*** *and converts them into* ***callback*** *invocations"*

codul JavaScript nu este executat paralel,
dar tratează în mod asincron diverse evenimente I/O

blog.mixu.net/2011/02/01/understanding-the-node-js-event-loop/

Aplicația Node.js „reacționează” la evenimente

context mai larg: *reactive programming* – **reactivex.io**



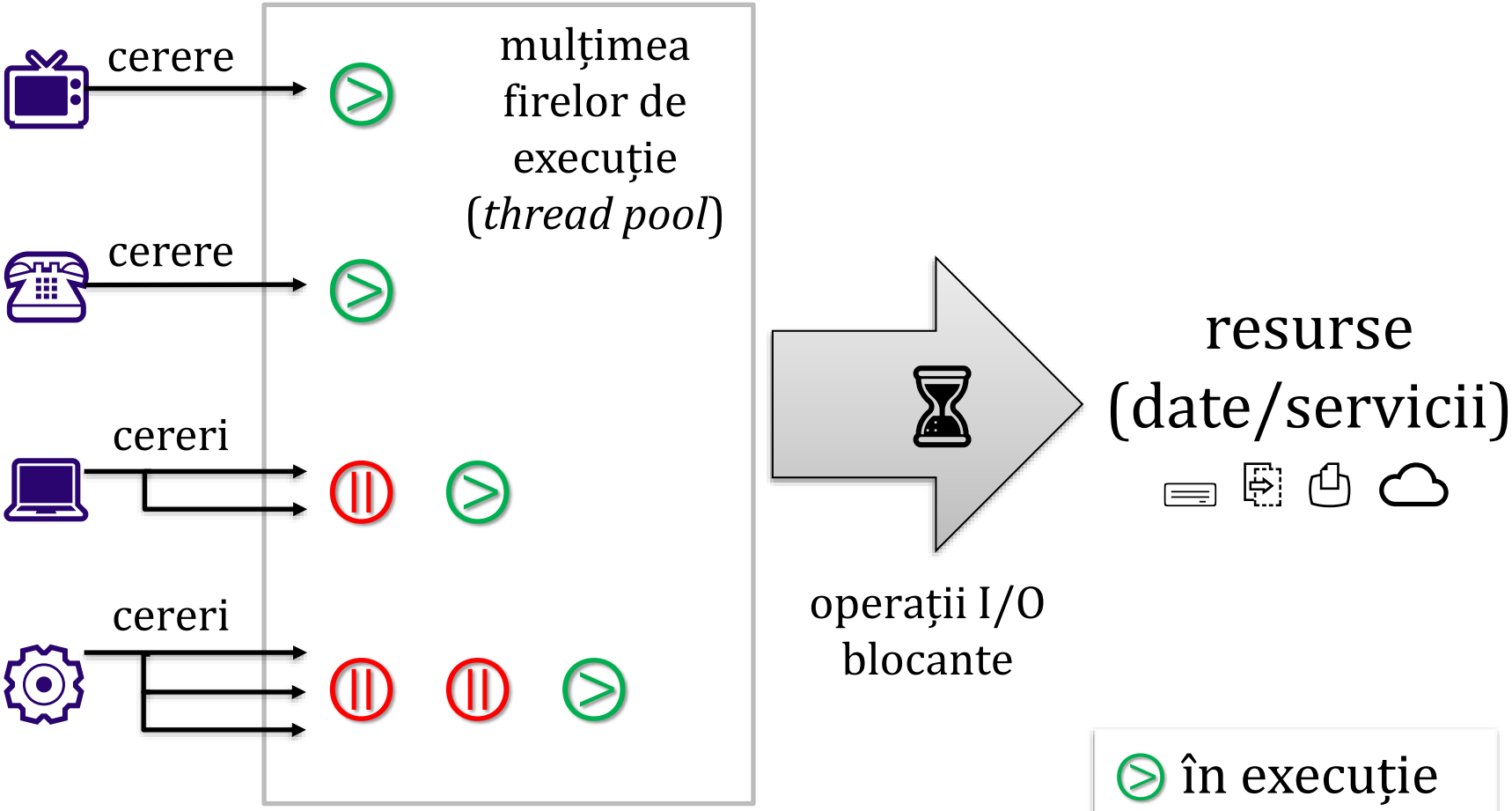
G. Yeh (2014): **www.slideshare.net/garyyeh165/basic-understanding-and-implement-of-nodejs**

node.js: caracterizare

O aplicație Node.js rulează într-**un singur proces**

deosebire esențială față de serverele de aplicații Web tradiționale ce recurg la servere *multi-process/threaded*

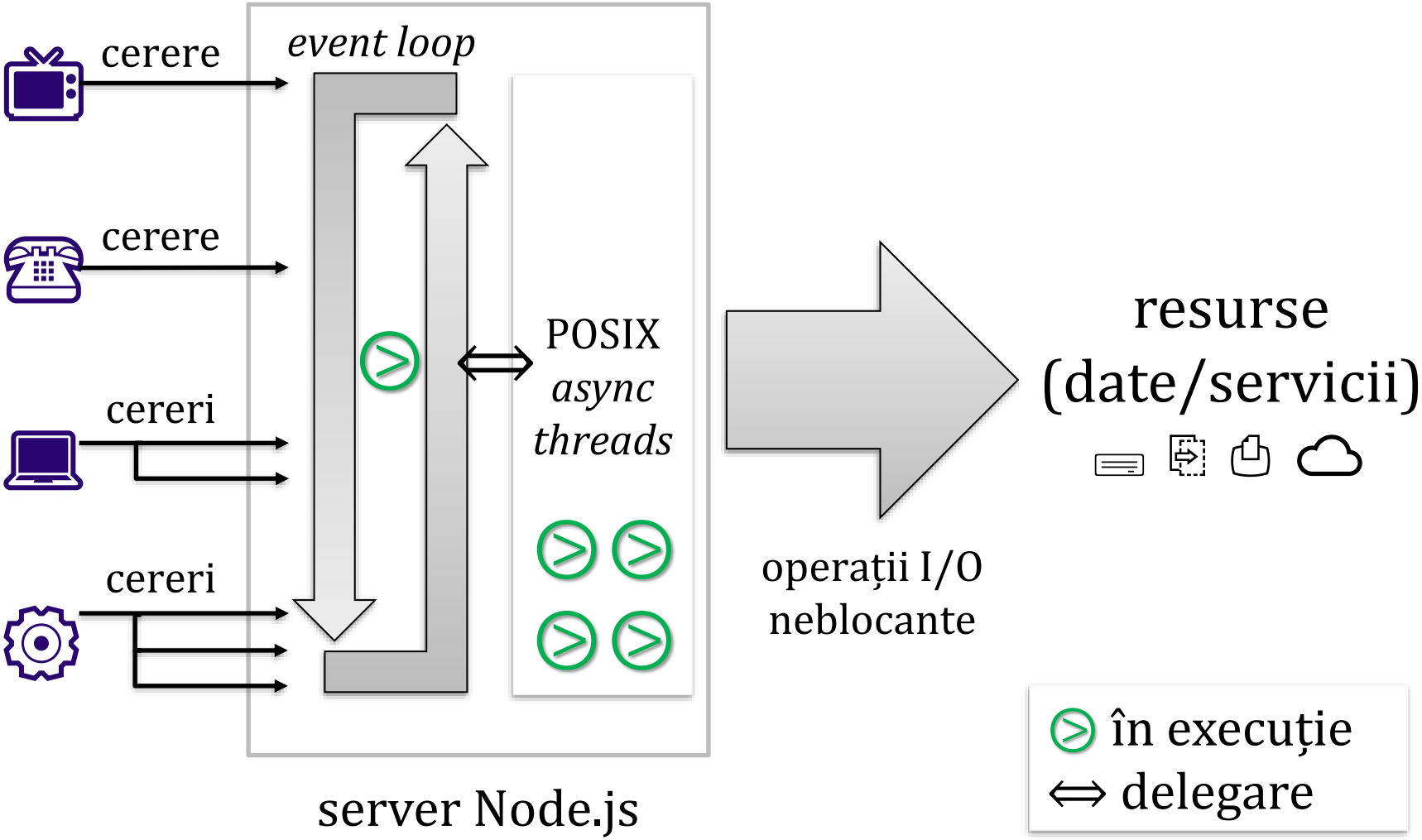
server Web clasic



server Web *multi-threaded*

cererile multiple de la diverși clienți nu pot fi deservite simultan (numărul firelor de execuție asociate unui proces este limitat)

Node.js



cererile multiple de la diverși clienți pot fi deservite simultan

node.js: caracterizare

Mediul Node.js e disponibil gratuit – *open source* – pentru platformele Linux, Windows, macOS

nodejs.org/en/download/

inclusiv imagini-sistem Docker: **hub.docker.com/_/node/**

Funcționalități suplimentare oferite de module administrate cu *Node Package Manager*

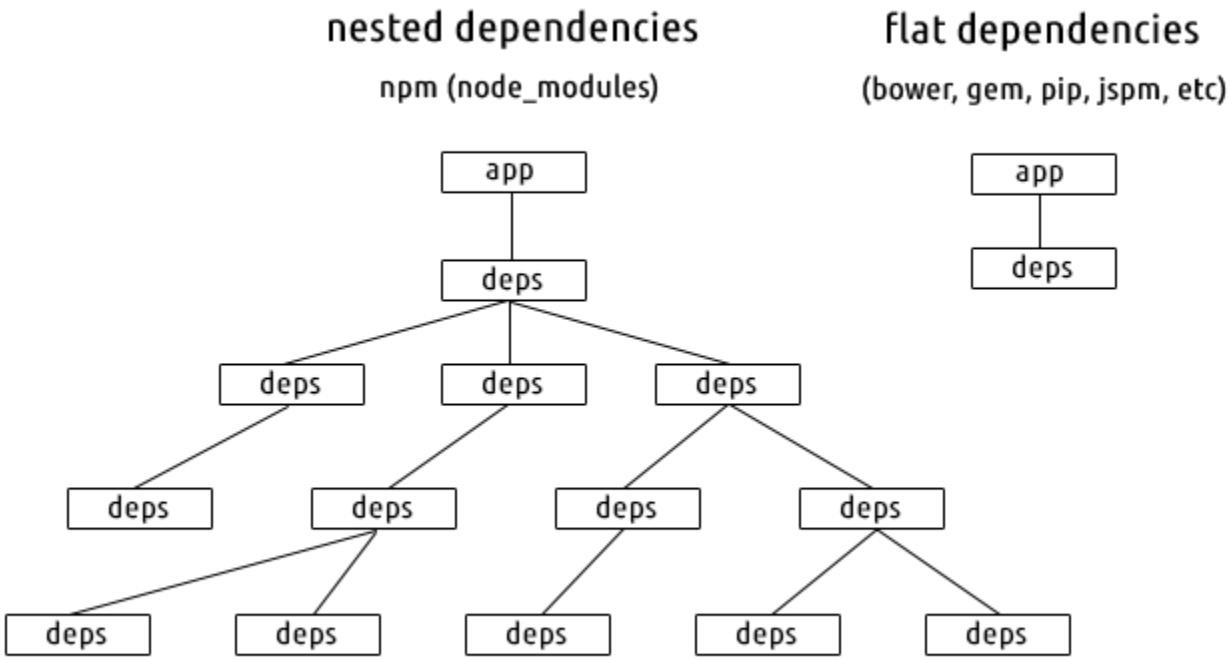
npm

utilitar în linia de comandă dezvoltat complet în JavaScript
(inițiat de Isaac Z. Schlueter în 2010)

npmjs.com
github.com/npm

node.js: caracterizare

Utilitarul **npm** poate gestiona dependențe imbricate

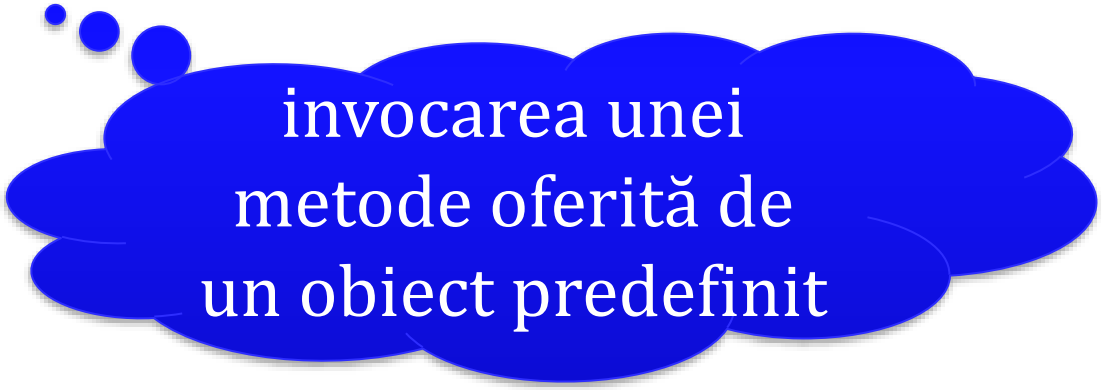


detalii la **docs.npmjs.com**

node.js: exemplu

Un prim program care emite mesaje de salut

```
// salutari.js: un program (de sine-stătător) care emite un salut
console.log ('Salutări banale din Node.js');
```



node.js: exemplu

Un prim program care emite mesaje de salut

```
// salutari.js: un program (de sine-stătător) care emite un salut
console.log ('Salutări banale din Node.js');
```

```
> node salutari.js
Salutări banale din Node.js
```

```
/* Un program JavaScript ilustrând comportamentul asincron
   al operațiilor I/O implementate de Node.js */
const fs = require ('fs');            // acces la sistemul de fișiere

fs.readFile ('salutari.txt', 'utf-8', function (eroare, date) {
  if (eroare)
    throw eroare;                     // excepție de citire
  console.log (date);
})
console.log ('Gata!');
```

execuția (cod interpretat) pornește
de la prima linie a programului JavaScript

```
/* Un program JavaScript ilustrând comportamentul asincron
   al operațiilor I/O implementate de Node.js */
const fs = require ('fs');            // acces la sistemul de fișiere

fs.readFile ('salutari.txt', 'utf-8', function (eroare, date) {
  if (eroare)
    throw eroare;                     // excepție de citire
  console.log (date);
})
console.log ('Gata!');
```

începe execuția unei operații asincrone
(aici, citirea conținutului unui fișier text)
care returnează imediat

```
/* Un program JavaScript ilustrând comportamentul asincron
   al operațiilor I/O implementate de Node.js */
const fs = require ('fs');            // acces la sistemul de fișiere

fs.readFile ('salutari.txt', 'utf-8', function (eroare, date) {
  if (eroare)
    throw eroare;                     // excepție de citire
  console.log (date);
})
console.log ('Gata!');
```

execuția continuă cu ultima linie de program

```
> node asincronism.js
Gata!
```

```
/* Un program JavaScript ilustrând comportamentul asincron
   al operațiilor I/O implementate de Node.js */
const fs = require ('fs');            // acces la sistemul de fișiere

fs.readFile ('salutari.txt', 'utf-8', function (eroare, date) {
  if (eroare)
    throw eroare;                     // excepție de citire
  console.log (date);
})
console.log ('Gata!');
```

...după care sunt afișate datele preluate din fișier

```
> node asincronism.js
Gata!
Un prim salut...
Acesta este al doilea salut.
```

```
// Un program JavaScript care salută toți posibilii săi clienți Web
const http = require ('http');      // folosim 'http', un modul Node predefinit

http.createServer (                 // creăm un server Web
  // funcție anonimă ce tratează o cerere și trimite un răspuns
  function (cerere, raspuns) {
    // afișăm la consola serverului mesaje de diagnostic
    console.log ('Am primit o cerere...');
    // stabilim valori pentru diverse câmpuri din antetul mesajului HTTP
    raspuns.writeHead (200, { 'Content-Type': 'text/html' });
    // emitem răspunsul propriu-zis conform tipului MIME (aici, cod HTML)
    raspuns.end ('<html><body><h1>Salutari…</h1></body></html>');
  }
  // serverul ascultă cereri la portul 8080 al mașinii locale
  ).listen (8080, "127.0.0.1");

console.log ('Serverul creat asteapta cereri la http://127.0.0.1:8080/');
```

programul JavaScript creat funcționează ca un server Web pentru fiecare cerere emisă de un posibil client (*browser*, aplicație *desktop* etc.) conform modelului client/server

pe partea de server – așteptare de cereri

```
> node salutari-web.js
Serverul creat asteapta cereri la http://127.0.0.1:8080/
Am primit o cerere...
Am primit o cerere...
```

programul JavaScript creat funcționează ca un server Web pentru fiecare cerere emisă de un posibil client (*browser*, aplicație *desktop* etc.) conform modelului client/server

pe partea de server – așteptare de cereri

```
> node salutari-web.js
Serverul creat asteapta cereri la http://127.0.0.1:8080/
Am primit o cerere...
Am primit o cerere...
```

la client – recepționarea răspunsului conform cererii GET emise de un program *desktop* și de un navigator Web

```
> node client-salutari.js
Am primit raspuns de la server -- cod HTTP: 200
Continut receptionat: <html><body>
<h1>Salutari din Node.js</h1></body></html>
```

```
// Un program JS care implementează un client pentru serviciul de salut
const http = require ('http');

http.get ('http://127.0.0.1:8080/',            // emite o cerere HTTP
  function (raspuns) {
    console.log ('Am primit raspuns de la server -- cod HTTP: '
      + raspuns.statusCode);                   // statusCode: 200, 404,...
  })
// tratăm diverse evenimente via funcții (eventual, anonime) de tip callback
.on ('error',                                  // eroare
  function (e) { console.log ('Eroare: ' + e.message); })
.on ('response',                               // receptare răspuns de la server
  function (raspuns) {                         // există date de procesat
    raspuns.on ('data', function (date) {
            console.log ('Continut receptionat: ' + date);
          });
  }
);
```

```
// Aceleași acțiuni realizate în ES6
// Implementează un client pentru serviciul de salut
const http = require ('http');

http.get ('http://127.0.0.1:8080/',          // emite o cerere HTTP
  raspuns => console.log ('Am primit raspuns de la server -- cod HTTP: '
                + raspuns.statusCode)        // statusCode: 200, 404,...
)
// tratăm diverse evenimente
.on ('error', e => console.log ('Eroare: ' + e.message)) // eroare
.on ('response',            // receptare răspuns de la server
     raspuns => {           // există date de procesat
       raspuns.on ('data',
                        date => console.log ("Continut receptionat:\n" + date));
       console.log ('Campuri-antet HTTP primite: '
                        + JSON.stringify(raspuns.headers, null, '\t'));
    }
);
```

```
> node client-salutari.js

Am primit raspuns de la server -- cod HTTP: 200
Campuri-antet HTTP primite: {
	"content-type": "text/html",
	"date": "Tue, 16 Mar 2021 08:12:32 GMT",
	"connection": "close",
	"transfer-encoding": "chunked"
}
Continut receptionat:
<html><body><h1>Salutari din Node.js</h1></body></html>
```

node.js: module

Funcția **require ()** specifică utilizarea unui modul Node.js

module predefinite (*built-in*) – exemple:

privitoare la tehnologii Web – **http**, **https**, **url**, **querystring**
referitoare la fișiere – **fs**, **path**
vizând rețeaua – **net**, **dns**, **dgram**, **tls**,...
resurse privind sistemul de operare – **os**, **child_process**
alte aspecte de interes – **buffer**, **console**, **util**, **crypto**
suport *multi-core* – **cluster**

Node.js

About these Docs
Usage & Example

Assertion Testing
Buffer
C/C++ Addons
Child Processes
Cluster
Command Line Options
Console
Crypto
Debugger
DNS
Domain
Errors
Events
File System
Globals
HTTP
HTTPS
Modules

Stability: 2 - Stable

The `child_process` module provides the ability to spawn child processes in a manner that is similar, but not identical, to `popen(3)`. This capability is primarily provided by the `child_process.spawn()` function:

```
const spawn = require('child_process').spawn;
const ls = spawn('ls', ['-lh', '/usr']);

ls.stdout.on('data', (data) => {
  console.log(`stdout: ${data}`);
});

ls.stderr.on('data', (data) => {
  console.log(`stderr: ${data}`);
});

ls.on('close', (code) => {
  console.log(`child process exited with code ${code}`);
});
```

documentația *online* aferentă

By default, pipes for `stdin`, `stdout` and `stderr` are established between the parent Node.js process and the spawned child. It is possible to stream data through these pipes in a non-blocking way. *Note, however, that some programs use line-buffered I/O internally. While that does not affect Node.js, it can mean that data sent to the child process may not be immediately consumed.*

nodejs.org/en/docs/
devdocs.io/node/

atenție: o parte dintre funcționalități sunt experimentale

Events
File System
Globals
HTTP
HTTP/2
HTTPS
Inspector
Internationalization
Modules

- Examples
  - Measuring the duration of async ope
  - Measuring how long it takes to load

Performance Timing API

Stability: 1 - Experimental

The Performance Timing API provides an implementation of the W3C Performance Timeline specification.

Stability: 0 - Deprecated This feature is known to be problematic, and changes may be planned. Do not rely on it. Use of the feature may cause warnings to be emitted. Backwards compatibility across major versions should not be expected.

Stability: 1 - Experimental This feature is still under active development and subject to non-backwards compatible changes, or even removal, in any future version. Use of the feature is not recommended in production environments. Experimental features are not subject to the Node.js Semantic Versioning model.

String Decoder

```
  performance.measure('A to B', 'A', 'B');
});
```

node.js: module – http

Dezvoltarea de aplicații Web via modulul **http**

funcționalități HTTP de bază

crearea unui server Web: **createServer()**

realizarea de cereri HTTP: **request() get()**

node.js: module – http

Dezvoltarea de aplicații Web via modulul **http**

servire de cereri HTTP – clasa **http.Server**

metode uzuale:
listen() setTimeout() close()

evenimente ce pot fi tratate:
request connect close clientError etc.

node.js: module – http

Dezvoltarea de aplicații Web via modulul **http**

răspuns emis de server – clasa **http.ServerResponse**

metode uzuale:
writeHead() getHeader() removeHeader() write() end() etc.

evenimente: **close finish**

proprietăți folositoare: **statusCode headersSent**

node.js: module – http

Dezvoltarea de aplicații Web via modulul **http**

cerere emisă de client – clasa **http.ClientRequest**

metode uzuale:
write() abort() end() setTimeout() setSocketKeepAlive()

evenimente ce pot fi tratate:
response connect continue socket etc.

node.js: module – http

Dezvoltarea de aplicații Web via modulul **http**

mesaj vehiculat – clasa **http.IncomingMessage**

metode: **setEncoding() setTimeout() pause() resume()**

evenimente ce pot fi tratate: **data end close**

proprietăți de interes:
httpVersion headers method url statusCode socket

node.js: module – http

Suplimentar, pot fi folosite modulele interne:

https – suport pentru HTTPS
nodejs.org/api/https.html

http2 – implementează HTTP/2
nodejs.org/api/http2.html

node.js: module – url

Procesarea adreselor Web via modulul **url**

clase oferite:

URL() URLSearchParams()

nodejs.org/api/url.html

```
let adresaWeb = new URL
  ('http://undeva.info:8080/oferta/jucarii/produs/?nume=Tux&marime=17#oferta');
adresaWeb.host = 'acolo';
adresaWeb.protocol = 'https';

console.log (adresaWeb);

// parametrii din query string (valoarea proprietății search)
// sunt încapsulați de obiectul URLSearchParams
let params = new URLSearchParams (adresaWeb.search);

if (params.get ('marime') > 13) {
  console.log ('Jucaria e in regula.');
} else {
  console.log ('Jucaria e stricata.');
}
```

```
> node url.js
URL {
  href:
   'https://acolo:8080/oferta/jucarii/produs/?nume=Tux&marime=17#oferta',
  origin: 'https://acolo:8080',
  protocol: 'https:',
  username: '',
  password: '',
  host: 'acolo:8080',
  hostname: 'acolo',
  port: '8080',
  pathname: '/oferta/jucarii/produs/',
  search: '?nume=Tux&marime=17',
  searchParams: URLSearchParams { 'nume' => 'Tux', 'marime' => '17' },
  hash: '#oferta' }
Jucaria e in regula.
```

node.js: module – net

Crearea de aplicații Internet – modulul **net**

partea de server:

createServer()

+

clasa **net.Server**

metode: **listen() close() address() getConnections()**

evenimente: **listening connection close error**

proprietăți: **listening maxConnections**

node.js: module – net

Crearea de aplicații Internet – modulul **net**

partea de client:

connect()
createConnection()

node.js: module – net

Crearea de aplicații Internet – modulul **net**

acces la *socket*-uri – clasa **net.Socket**

metode: **connect() write() setEncoding() destroy() end()** etc.

evenimente: **connect data end timeout drain error close**

proprietăți utile: **localAddress localPort remoteAddress remotePort bytesRead bytesWritten bufferSize**

node.js: module – fs

Acces la sistemul de fișiere via modulul **fs**

clase importante:

fs.ReadStream – flux de citire

fs.WriteStream – flux de scriere

fs.Stats – informații despre un fișier

fs.FSWatcher – monitorizează schimbările dintr-un fișier (obiecte de tip EventEmitter – evenimentul change)

fs.Dirent – element dintr-un director (*directory entry*)

node.js: module – fs

Acces la sistemul de fișiere via modulul **fs**

metode folosite uzual – comportament asincron:

open() read() write() close()

truncate() stat() chmod() rename() unlink() watch()

isFile() isDirectory() isBlockDevice() isSocket()

mkdir() rmdir() readdir()

node.js: module – fs

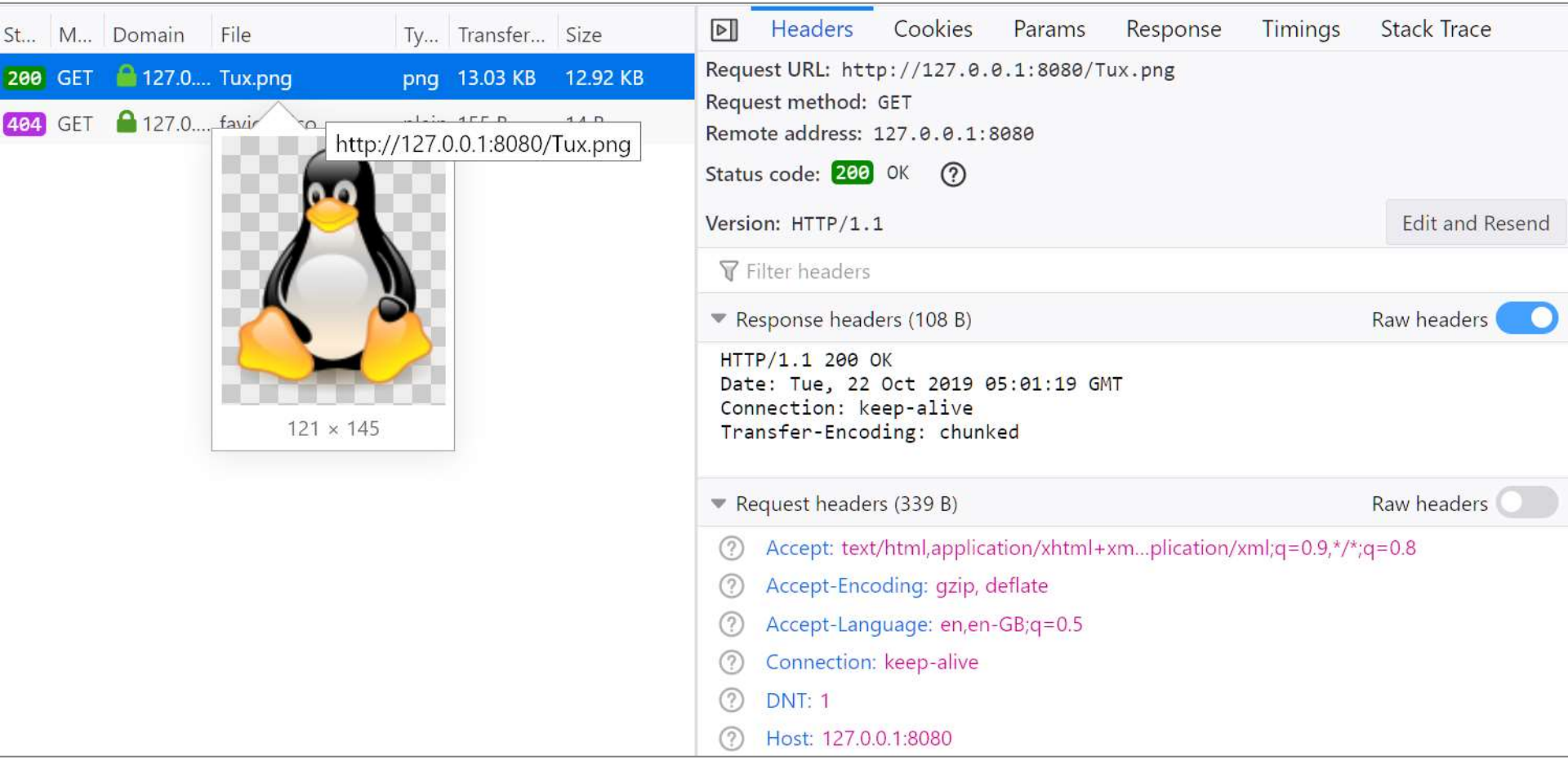
Acces la sistemul de fișiere via modulul **fs**

unele metode au comportament sincron
(numele lor sunt sufixate cu Sync)

exemple:
openSync() readSync() writeSync() closeSync()
appendFileSync() chmodSync() unlinkSync() mkdirSync()

mini-server Web oferind fișiere statice stocate în 'public/'

```
const http  = require ("http");
const url   = require ("url");
const path = require ("path");
const fs    = require ("fs");

http.createServer ((cerere, raspuns) => {
   let uri = url.parse (cerere.url).pathname; // preluăm URL și calea spre fișierul dorit
   let numefis = path.join (process.cwd (), 'public/', uri);
   fs.access (numefis, fs.constants.R_OK, (eroare) => {
      if (eroare) {                    // resursa nu există, trimitem bine-cunoscutul cod 404
         raspuns.writeHead (404, { "Content-Type": "text/plain" });
         raspuns.write ("404 Not Found\n");
         raspuns.end ();               // închidem fluxul de date spre client
         return;
      }
   fs.readFile (numefis, "binary", (eroare, fisier) => { // resursa există, poate fi citită
      if (eroare) {                    // eroare de citire a fisierului, trimitem 500
         raspuns.writeHead (500, { "Content-Type": "text/plain" });
         raspuns.write (eroare + "\n"); raspuns.end (); return;
      }
      raspuns.writeHead (200); // totul e în regulă, trimitem fișierul ca flux binar
      raspuns.write (fisier, "binary");
      raspuns.end ();
      });
   }); }).listen (8080);

console.log ('Serverul de fișiere așteaptă cereri la http://127.0.0.1:8080/');
```



cererea GET de acces a reprezentării resursei de la http://127.0.0.1:8080/Tux.png conduce la obținerea unei imagini în format PNG (aici, inspecția transferului datelor cu instrumentele oferite de *browser*)

node.js: fluxuri de date

Accesul la date poate fi realizat
prin intermediul fluxurilor (***streams***)

abstractizează accesul la date stocate parțial
(*partially buffered data*)

se emit evenimente ce pot fi tratate de codul aplicației

node.js: fluxuri de date

Accesul la date poate fi realizat
prin intermediul fluxurilor (***streams***)

pot fi citite – **stream.Readable**
pot fi scrise – **stream.Writable**
duplex (citite și scrise) – **stream.Duplex**
realizând o transformare a datelor – **stream.Transform**

detalii la **nodejs.org/api/stream.html**

Readable Streams	Writable Streams
HTTP response (client)	HTTP requests (client)
HTTP requests (server)	HTTP responses (server)
fs read streams	fs write streams
zlib streams	zlib streams
crypto streams	crypto streams
TCP sockets	TCP sockets
child process stdout & stderr	child process stdin
process.stdin	process.stdout process.stderr

jscomplete.com/learn/node-beyond-basics/node-streams

node.js: fluxuri de date

Accesul la date poate fi realizat
prin intermediul fluxurilor (***streams***)

fluxuri ce pot fi citite (*readable streams*)
e.g., create de **fs.createReadStream()** **http.ServerRequest**
http.ClientResponse **net.Socket** **child.stdout** **process.stdin**

node.js: fluxuri de date

Accesul la date poate fi realizat
prin intermediul fluxurilor (***streams***)

fluxuri ce pot fi citite (*readable streams*)
e.g., create de **fs.createReadStream()** **http.ServerRequest**
http.ClientResponse **net.Socket** **child.stdout** **process.stdin**

emit evenimentele **readable data end error**

node.js: fluxuri de date

Accesul la date poate fi realizat
prin intermediul fluxurilor (***streams***)

fluxuri ce pot fi citite (*readable streams*)
e.g., create de **fs.createReadStream() http.ServerRequest**
http.ClientResponse net.Socket child.stdout process.stdin

au asociate metodele **pause() resume() destroy()**

node.js: fluxuri de date

Accesul la date poate fi realizat
prin intermediul fluxurilor (***streams***)

fluxuri ce pot fi scrise (*writeable streams*)
e.g., create de **fs.createWriteStream()** **http.ServerResponse**
http.ClientRequest **net.Socket** **child.stdin** **process.stdout**

node.js: fluxuri de date

Accesul la date poate fi realizat
prin intermediul fluxurilor (***streams***)

fluxuri ce pot fi scrise (*writeable streams*)
e.g., create de **fs.createWriteStream()** **http.ServerResponse**
http.ClientRequest **net.Socket** **child.stdin** **process.stdout**

emit evenimentele **drain error**

node.js: fluxuri de date

Accesul la date poate fi realizat
prin intermediul fluxurilor (***streams***)

fluxuri ce pot fi scrise (*writeable streams*)
e.g., create de **fs.createWriteStream()** **http.ServerResponse**
http.ClientRequest **net.Socket** **child.stdin** **process.stdout**

oferă metodele **write() end() destroy()**

```
// Program ce preia ceea ce tastează utilizatorul la intrarea standard
// și scrie într-un fișier – adaptare după M. Takada (2012)
var fs = require ('fs');

var fisier = fs.createWriteStream ('./spion.txt');

// la apariția datelor, le scriem în fișier
process.stdin.on ('data', date => fisier.write (date));

// tratăm evenimentul de terminare a fluxului
process.stdin.on ('end', () => fisier.end ());

// "reactivăm" intrarea standard; implicit, e în starea 'paused'
process.stdin.resume ();
```

obiectul **process** e global – detalii la **nodejs.org/api/process.html**

node.js: fluxuri de date

Accesul la date poate fi realizat
prin intermediul fluxurilor (***streams***)

exemple de fluxuri de tip duplex:
socket-uri TCP create cu **net.Socket()**
privind arhivele create cu **zlib** – **nodejs.org/api/zlib.html**
date criptate via **crypto** – **nodejs.org/api/crypto.html**

node.js: rulare temporizată

Se poate planifica execuția codului JavaScript

recurgerea la funcțiile globale
setTimeout () clearTimeout () – execuție la un anumit moment
setInterval () clearInterval () – execuție recurentă
setImmediate () clearImmediate () – execuție „imediată”

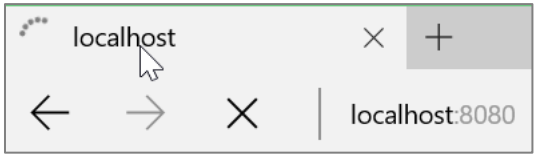
amănunte la **nodejs.org/api/timers.html**

```
// creăm un server Web care trimite fiecărui client secvența valorilor unui contor
var server = http.createServer ().listen (8080, '127.0.0.1');

// stabilim un comportament la apariția evenimentului 'request' (cerere de la un client)
server.on ('request', function (cerere, raspuns) {
  console.log ('Cerere de la clientul ' + cerere.headers['user-agent']);
  raspuns.writeHead (200, { 'Content-Type': 'text/html' });

  var contor = 0;
  var interval = setInterval ( // generăm valori ale contorului conform intervalului de timp
    function () {
      raspuns.write ('<p>Contorul are valoarea ' + contor + '</p>');
      console.log ('Contorul are valoarea ' + contor);
      contor++;
      if (contor >= 7) {
         clearInterval (interval);            // ștergem intervalul
         raspuns.end ();                      // închidem fluxul de răspuns
         console.log ('Am trimis raspuns clientului ' + cerere.headers['user-agent']);
      }
    }, 1000);                                 // cod rulat la interval de 1000 milisecunde
});
```

Cerere de la clientul … Mozilla/5.0 (Windows NT 10.0 …) … Firefox/86.0
Contorul are valoarea 0
Contorul are valoarea 1
Contorul are valoarea 2
Contorul are valoarea 3
Cerere de la clientul … Edge/18.18362
Contorul are valoarea 4
Contorul are valoarea 0
Contorul are valoarea 5
Contorul are valoarea 1
Contorul are valoarea 6
Am trimis raspuns clientului … Mozilla/5.0 (Windows NT 10.0 …) … Firefox/86.0
Contorul are valoarea 2
Contorul are valoarea 3
Contorul are valoarea 4
Contorul are valoarea 5
Contorul are valoarea 6
Am trimis raspuns clientului … Edge/18.18362

codul este rulat asincron

Cerere de la clientul … Mozilla/5.0 (Windows NT 10.0 …) … Firefox/86.0
Contorul are valoarea 0
Contorul are valoarea 1
Contorul are valoarea 2
Contorul are valoarea 3
Cerere de la clientul … Edge/18.18362
Contorul are valoarea 4
Contorul are valoarea 0
Contorul are valoarea 5
Contorul are valoarea 1
Contorul are valoarea 6
Am trimis raspuns clientului … Mozilla/5.0 (Windows NT 10.0 …) … Firefox/86.0
Contorul are valoarea 2
Contorul are valoarea 3
Contorul are valoarea 4
Contorul are valoarea 5
Contorul are valoarea 6
Am trimis raspuns clientului … Edge/18.18362



browser-ul Web va aștepta ca întreaga secvență de valori să fie trimisă de către server

de ce?

node.js: evenimente

Emiterea (lansarea) și tratarea (prinderea) evenimentelor specificate de programator se realizează via **event.EventEmitter**

clasă utilizată intern de multe biblioteci de bază

nodejs.org/api/events.html

```
const EventEmitter = require('events');
const cataclism = Symbol('©©©');

class Emițător extends EventEmitter {}

const emițător = new Emițător(); // un emițător de evenimente

// tratăm evenimentele 'bum' și 'cataclism'
emitator.on('bum', () => { console.log('Bum, bum, bum...'); });
emitator.on('cataclism', (e) => { console.error('Cataclism! ' + e.message); });

// emitem evenimente (proprii și deja existente)
try {
  emițător.emit('bum');
  emițător.emit('cataclism', new Error(cataclism.toString()));
  emițător.emit('error');
} catch (e) {
  console.error ('A survenit o eroare... ' + e.message);
}
```

```
const EventEmitter = require('events');
const cataclism = Symbol('©©©');

class Emițător extends EventEmitter {}

const emițător = new Emițător(); // un emițător de evenimente

// tratăm evenimentele 'bum' și 'cataclism'
emitator.on('bum', () => { console.log('Bum, bum, bum...'); });
emitator.on('cataclism', (e) => { console.error('Cataclism! ' + e.message); });

// emitem evenimente (proprii și deja existente)
try {
  emițător.emit('bum');
  emițător.emit('cataclism', new Error(cataclism.toString()));
  emițător.emit('error');
} catch (e) {
  console.error ('A survenit o er
}
```

```
> node emitere-evenim.js
Bum, bum, bum...
Cataclism! Symbol(©©©)
A survenit o eroare... Unhandled error. (undefined)
```

node.js: module externe

Funcția **require ()** specifică utilizarea unui modul Node.js

module disponibile *on-line*
(instalate și gestionate via utilitarul **npm**)

tutorial:
www.sitepoint.com/beginners-guide-node-package-manager/

node.js: module externe

Funcția **require ()** specifică utilizarea unui modul Node.js

instalare globală a unui modul: **npm install *modul* –g**
listarea modulelor ce sunt instalate local: **npm list**
căutarea unui modul: **npm search *modul***
eliminarea unui modul: **npm uninstall *modul***
actualizarea unui modul: **npm update *modul***

amănunte la **docs.npmjs.com**

node.js: module – acces la baze de date

Operații cu baze de date relaționale SQLite

recurgerea la modulul **sql.js** via compilatorul **emscripten**

utilizează o bază de date virtuală stocată în memoria RAM
(importă fișiere SQLite, exportă obiecte *typed array*)

detalii la **www.npmjs.com/package/sql.js**

node.js: module – acces la baze de date

Operații cu baze de date relaționale SQLite

sql.js nu depinde de alte module

actualmente, implementat în WebAssembly (WASM)

exemple demonstrative (interpretor SQL ca aplicație Web):
kripken.github.io/sql.js/examples/GUI/

node.js: module – acces la baze de date

Operații cu baze de date relaționale

Sequalize

acces abstractizat via ORM (*Object-Relational Mapping*) pentru servere relaționale: Postgres, MySQL, MariaDB, SQLite și Microsoft SQL Server

sequelize.org

node.js: module – acces la baze de date

Operații cu baze de date relaționale

alternative ORM la **Sequalize**:

Objection.js – **vincit.github.io/objection.js/**
ghid: **dev.to/mrscx/a-definitive-guide-to-sql-in-nodejs-with-objection-js-knex-part-1-4c2e**

Bookshelf.js – **bookshelfjs.org**
tutorial: **zetcode.com/javascript/bookshelf/**

node.js: module – prelucrări HTML

Procesarea documentelor HTML

modulul **JSDOM** conceput integral în JavaScript

simulează comportamentul unui *browser* Web
și are suport pentru execuție de *script*-uri JS
- util pentru testare și *scraping*

github.com/jsdom/jsdom

```
const jsdom = require("jsdom");
const { JSDOM } = jsdom; // acces la obiectul JSDOM oferit de modul

const URL = "https://profs.info.uaic.ro/~busaco/teach/courses/web/index.html";

JSDOM.fromURL(URL, { storageQuota: 5000 }) // promite să preia datele
.then(dom => {
  console.log
    (dom.window.document.body.querySelector("p").textContent);
  console.log(dom.serialize());
})
.catch(eroare => console.error ('Eroare! ' + eroare.message));
```

```
const jsdom = require("jsdom");
const { JSDOM } = jsdom; // acces la obiectul JSDOM oferit de modul

const URL = "https://profs.info.uaic.ro/~busaco/teach/courses/web/index.html";

JSDOM.fromURL(URL, { storageQuota: 5000 }) // promite să preia datele
.then(dom => {
  console.log
    (dom.window.document.body.querySelector("p").textContent);
  console.log(dom.serialize());
})
.catch(eroare => console.error ('Eroare! ' + eroare.message));
```

```
> node mini-robot.js
fundația unui viitor dezvoltator Web profesionist
```

node.js: module – prelucrări HTML

Procesarea documentelor HTML

alternative la **JSDOM**:

htmlparser2 – procesor HTML/XML fără validare
github.com/fb55/htmlparser2

node-html-parser – axat asupra vitezei (DOM simplificat)
github.com/taoqf/node-html-parser

node.js: module – prelucrări XML

Procesarea documentelor XML

fast-xml-parser – procesor cu validare, XML↔JSON
github.com/NaturalIntelligence/fast-xml-parser

libxmljs – portarea bibliotecii LibXML
github.com/libxmljs/libxmljs

parse-xml – fără suport pentru spații de nume și altele
github.com/rgrove/parse-xml

Aspecte vizând arhitectura și ingineria aplicațiilor Node.js?

node.js: *multithreading*

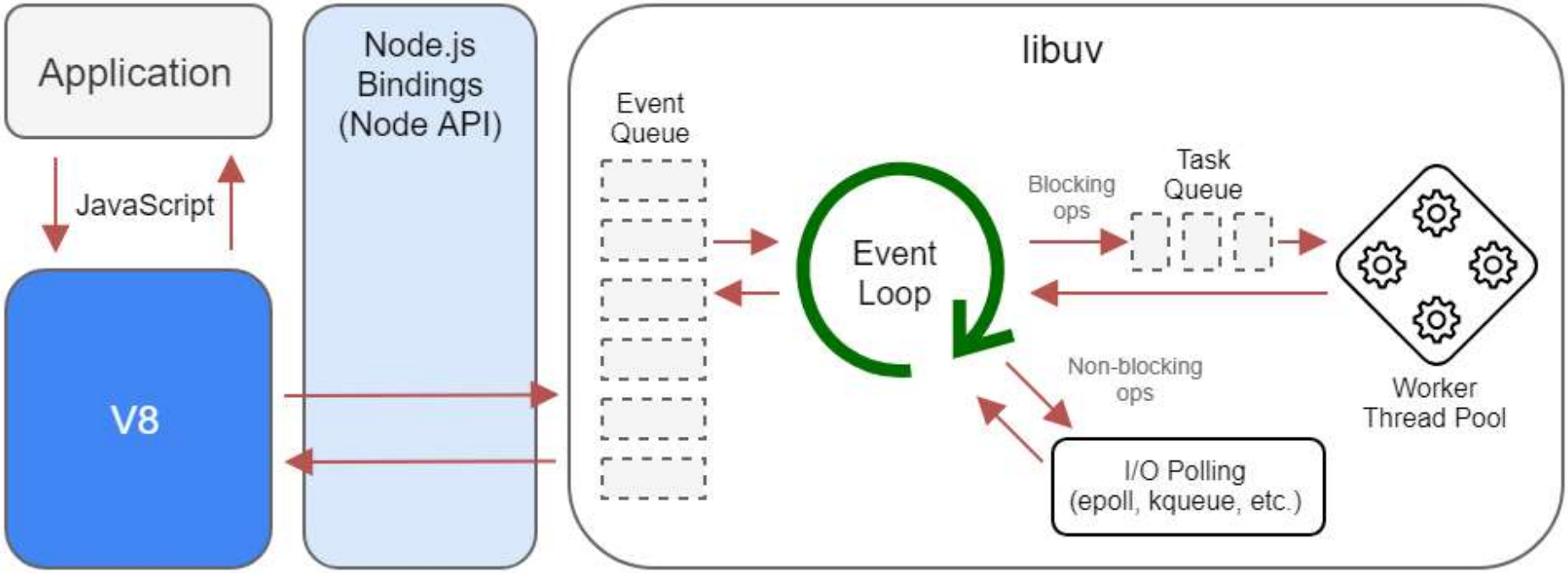
Actualmente (Node.js 12+), există suport
pentru fire de execuție multiple via modulul **worker_threads**

utilizări tipice: operații de calcul îndelungate
(*CPU-intensive operations*)

nodejs.org/api/worker_threads.html

medium.com/lazy-engineering/node-worker-threads-b57a32d84845

event loop + worker thread pool



activitățile consumatoare de timp pot fi efectuate de „lucrători” (*workers*) – programe JS neinteractive

itnext.io/an-intro-to-node-js-that-you-may-have-missed-b175ef4277f7

node.js: pachete

Un proiect (aplicație) Node.js compus(ă) din fișiere multiple poate fi organizat ca un pachet/modul

node.js: pachete

package.json

meta-date (numele pachetului, versiune, autor,...) + informații privind dependențele de alte module

docs.npmjs.com/creating-a-package-json-file

```
{
  "name": "WebApp",
  "version": "1.0.0",
  "main": "index.js",
  "dependencies": {
    "xml": "^1.0.1",
    "another_dep": "~2.2.0"
  },
  "devDependencies" : {
    "test_framework": "1.0.0 - 1.2.0"
  }
  "keywords": [ "xml", "json" ],
  "license": "BSD-2-Clause",
  "author": {
    "name": "Tuxy Pinguinnesscool",
    "email": "tux@penguin.info"
  },
  "description": "…"
}
```

main specifică numele fișierului (uzual, **index.js**) ce va fi încărcat când modulul este solicitat de altă aplicație

poate reprezenta programul principal, executat primul de mediul de rulare

```
{
  "name": "WebApp",
  "version": "1.0.0",
  "main": "index.js",
  "dependencies": {
    "xml": "^1.0.1",
    "another_dep": "~2.2.0"
  },
  "devDependencies" : {
    "test_framework": "1.0.0 - 1.2.0"
  }
  "keywords": [ "xml", "json" ],
  "license": "BSD-2-Clause",
  "author": {
    "name": "Tuxy Pinguinnesscool",
    "email": "tux@penguin.info"
  },
  "description": "..."
}
```

dependențele de alte module se declară via **dependencies** + aspecte vizând versiunea

docs.npmjs.com/about-semantic-versioning

devDependencies include dependențele necesare doar în faza de dezvoltare

node.js: pachete

Un pachet poate avea un domeniu de vizibilitate (*scope*)
docs.npmjs.com/about-scopes

package.json poate fi generat automat cu **npm init** sau
npm init --scope=@*organizație*

node.js: pachete

Pachetele care n-au specificat domeniul de vizibilitate sunt considerate automat publice

pachetele private trebuie să aibă obligatoriu asociat un domeniu de vizibilitate
(*private packages are always scoped*)

node.js: pachete

Pentru publicarea în registrul *on-line* al modulelor NPM se folosește **npm publish**

docs.npmjs.com/cli/publish

node.js: pachete

package-lock.json

generat automat când se modifică arborele de module (**node_modules**) sau fișierul **package.json**

descrie arborele curent de dependențe al unui proiect pentru a facilita instalarea și mentenanța

docs.npmjs.com/files/package-lock.json

Online Nodejs compiler, Online Nodejs ...
Code Nodejs, compile Nodejs, run Nodejs, and host your prog...

save | run | + new repl | Sign up

Files

- index.js
- package-lock.json
- package.json

index.js — saved

```
1  const xml = require('xml');
2
3  let echipa = [{ echipa: [{ _attr: { proiect: 'STAW', desc: '<?>' } }, {
   membru: '😵' }, { membru: '😱' }] }];
4  let echipaXML = xml(echipa, true); // JSON -> XML
5
6  console.log(echipaXML);
7
```

vezi arhiva exemplelor

```
node v10.16.0
>
<echipa proiect="STAW" desc="&lt;?&gt;">
    <membru>😵</membru>
    <membru>😱</membru>
</echipa>
```

editarea, organizarea și rularea *on-line* a codului Node.js cu instrumentul Web REPL.it – **repl.it/languages/nodejs**

aici, un program ce transformă date JSON în format XML pe baza modulului extern xml

node.js: *framework*-uri web

Uzual, încurajează dezvoltarea de aplicații Web
în care interacțiunea cu utilizatorul
se realizează într-o singură pagină
(SPA – *Single-Page Applications*)

real-time single-page Web apps

Paul Sherman, *How Single-Page Applications Work*
blog.pshrmn.com/how-single-page-applications-work/

Full-stack frameworks

That's where Node.js really shines. The full-stack MVC frameworks are bundled with scaffolding, template engines, websocket and persistence libraries to allow you build real-time scalable web apps.

- Meteor
- Ghost
- Strapi
- Keystone.js
- MEAN.io
- Feathers
- Horizon
- SocketCluster
- MEAN.js
- Derby
- SocketStream

alte facilități notabile:
MVC (*Model-View-Controller*) și variantele
transfer de date în timp-real – *e.g.*, cu Socket.IO
servicii Web – REST și/sau GraphQL
suport pentru baze de date (No)SQL
machete de redare a conținutului (*templates*)

lista celor mai semnificative *framework*-uri Node.js
nodeframework.com

node.js: *framework*-uri web – express

Framework minimalist
ce încurajează dezvoltarea de aplicații Web

tradiționale – reprezentări multiple via hipermedia
bazate pe un unic punct de acces: SPA (*Single Page App*)
hibride (Web + aplicații native)

expressjs.com
developer.mozilla.org/docs/Learn/Server-side/Express_Nodejs

```
.
├── app.js
├── bin
│   └── www
├── package.json
├── public
│   ├── images
│   ├── javascripts
│   └── stylesheets
│       └── style.css
├── routes
│   ├── index.js
│   └── users.js
└── views
    ├── error.pug
    ├── index.pug
    └── layout.pug
```

conținut static destinat clientului (*e.g.*, foi de stiluri CSS, biblioteci JS procesate de *browser*, imagini,...)

rute vizând deservirea cererilor pe baza URL-urilor solicitate de client

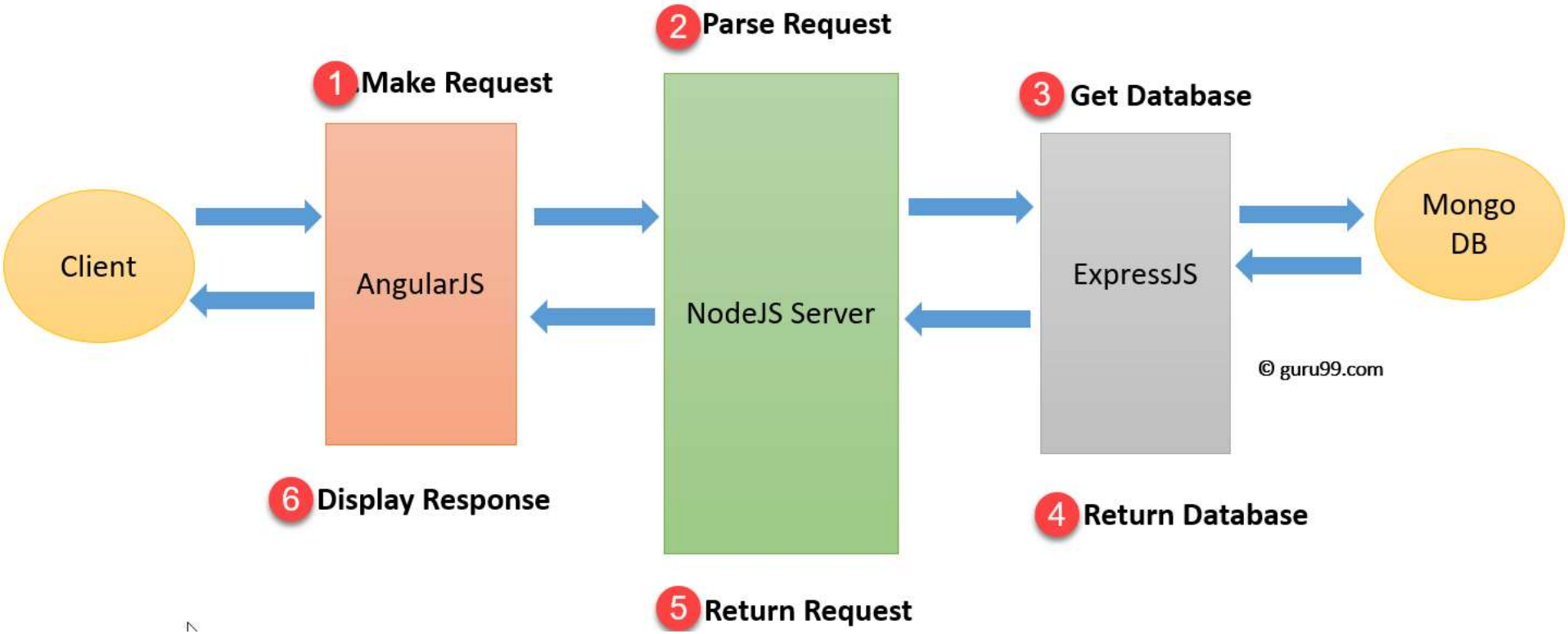
specificarea interfeței (*view*-ul) via machete de vizualizare bazate pe Pug: **pugjs.org**

eșafodajul unei aplicații Web bazate pe Express
create cu **express** **--view=pug** ***web-project***

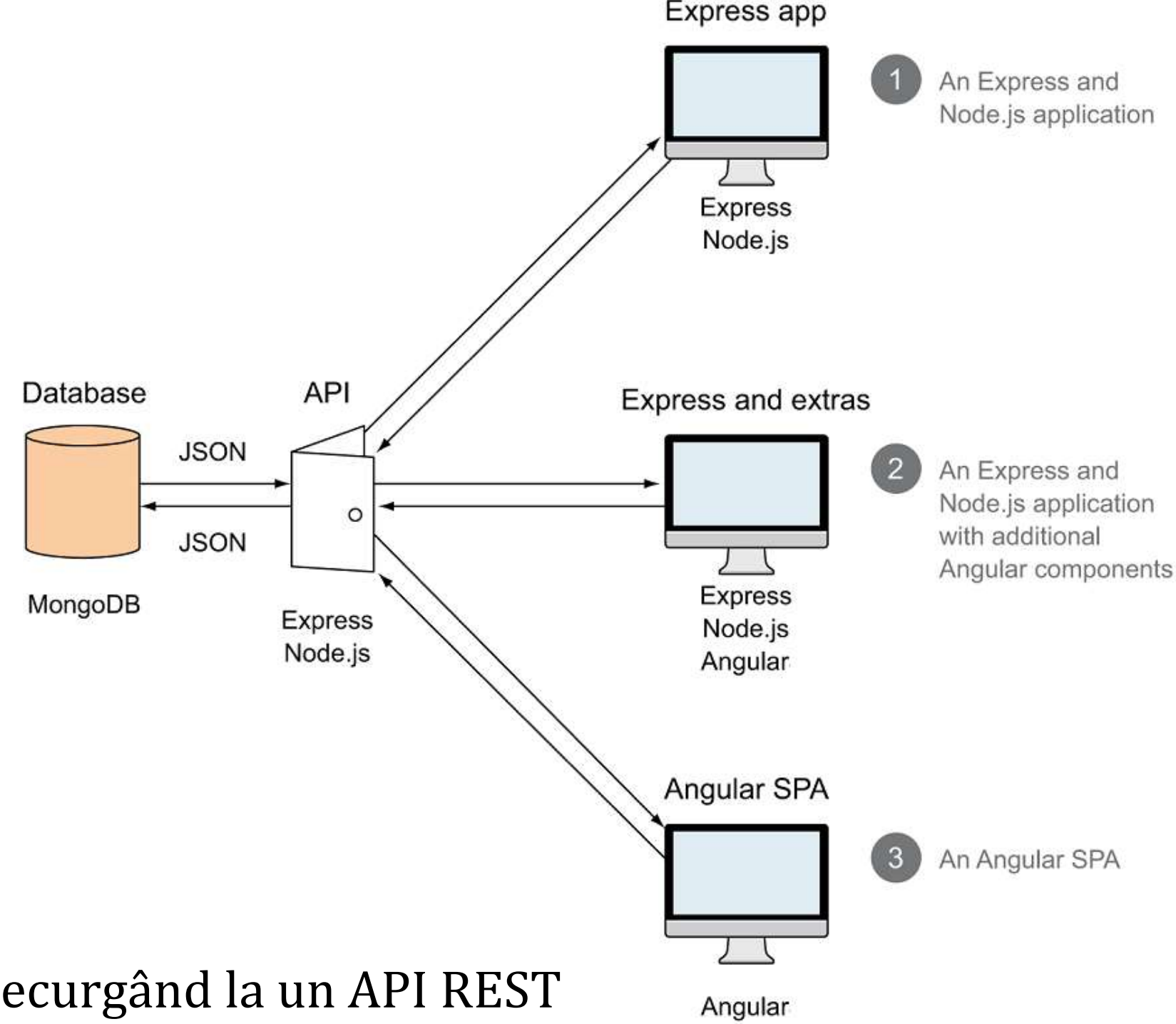
implicit, aplicația va putea fi accesată la **http://localhost:3000/**

node.js: *framework*-uri web

MEAN (MongoDB, Express, Angular, Node.js)



thinkster.io/tutorials/mean-stack/jumping-in-with-angular
www.guru99.com/mean-stack-developer.html



MEAN recurgând la un API REST

node.js: *framework*-uri web

MERN (MongoDB, Express, React, Node.js)

tutoriale:

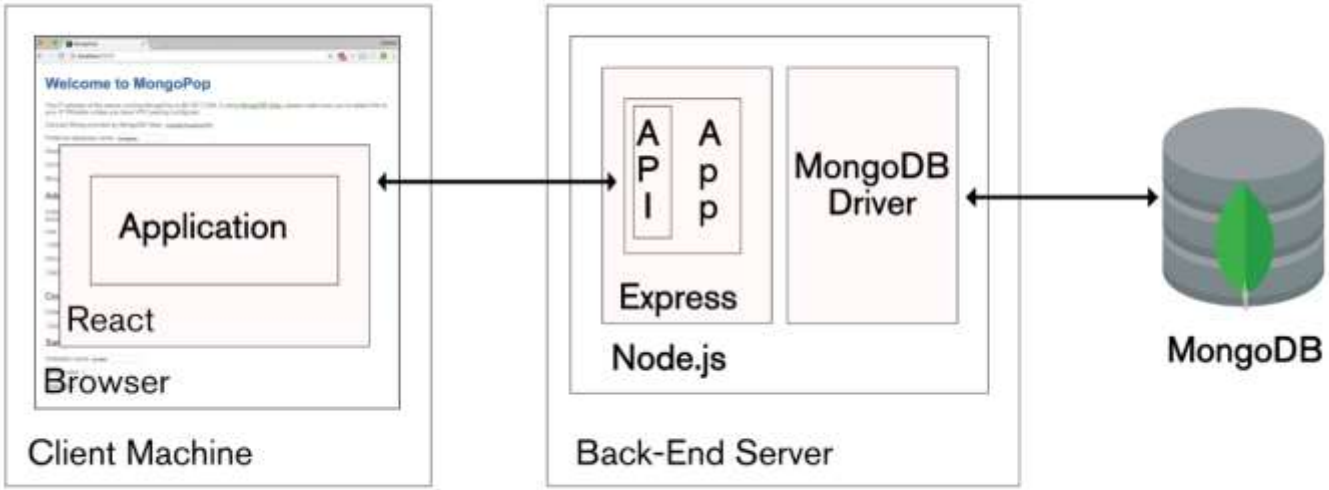
MERN Stack

www.mongodb.com/mern-stack

MERN stack A to Z

blog.logrocket.com/mern-stack-tutorial/

node.js: *framework*-uri web



pentru acces asincron la serverul MongoDB
se poate folosi modulul **Mongoose**
mongoosejs.com

node.js: *framework*-uri web

Alternative/specializări:

Adonis – **adonisjs.com/docs/**
Fastify – **www.fastify.io/docs/latest/**
Koa.js – **github.com/koajs**
Loopback – **loopback.io/doc/**
Nanoexpress – **github.com/nanoexpress**
NestJS – **docs.nestjs.com**
Sails – **sailsjs.com/documentation/concepts**

node.js: *framework*-uri web

Implementarea (micro-)serviciilor Web

ghiduri practice:

Building JavaScript Microservices with Node.js

www.twilio.com/blog/building-javascript-microservices-node-js

REST APIs with Node.js, MongoDB, Fastify and Swagger

tinyurl.com/y3t6br3m

node.js: utilizări pragmatice

Deservirea unui volum mare de conexiuni concurente cu necesar minim de resurse (procesor, memorie) într-un singur proces

node.js: utilizări pragmatice

Procesarea în timp-real
a datelor JSON oferite de API-uri (multiple)

inclusiv, crearea de aplicații
oferind fluxuri de date (*streaming data*)

node.js: utilizări pragmatice

Procesarea în timp-real
a datelor JSON oferite de API-uri (multiple)

studii de caz:
www.pubnub.com/blog/tag/node-js/

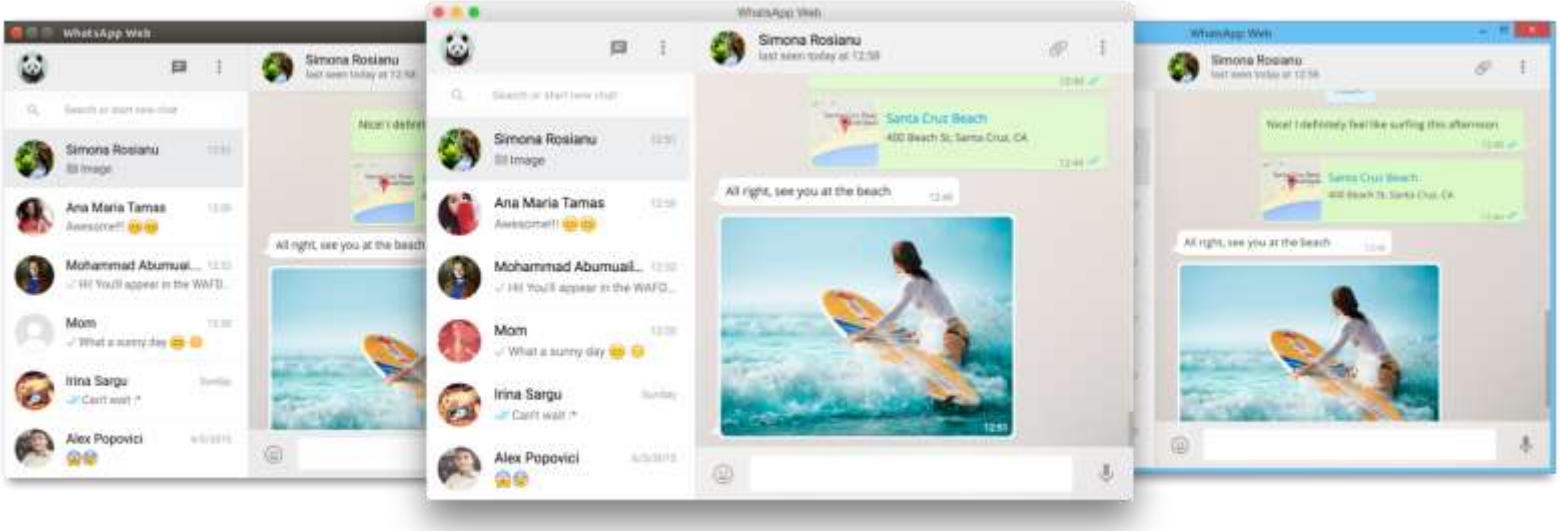
node.js: utilizări pragmatice

Dezvoltarea rapidă de servicii Web
sau API-uri conform paradigmei REST
(*REpresentational State Transfer*) ori GraphQL

framework-uri pentru dezvoltarea de API-uri – exemple:
egg, Hapi, LoopBack, NestJS,
Restgoose, restify, TypeGraphQL

node.js: utilizări pragmatice

Crearea de aplicații native (*desktop*)
folosind tehnologii Web moderne precum HTML5



NW.js (fost node-webkit) – **nwjs.io**
utilizarea modulelor Node.js
direct la nivel de DOM via Chromium

node.js: instrumente de dezvoltare

Mediu de dezvoltare tradițional (*desktop*)

exemplificări:

Node.js Tools for Visual Studio

visualstudio.microsoft.com/vs/features/node-js/

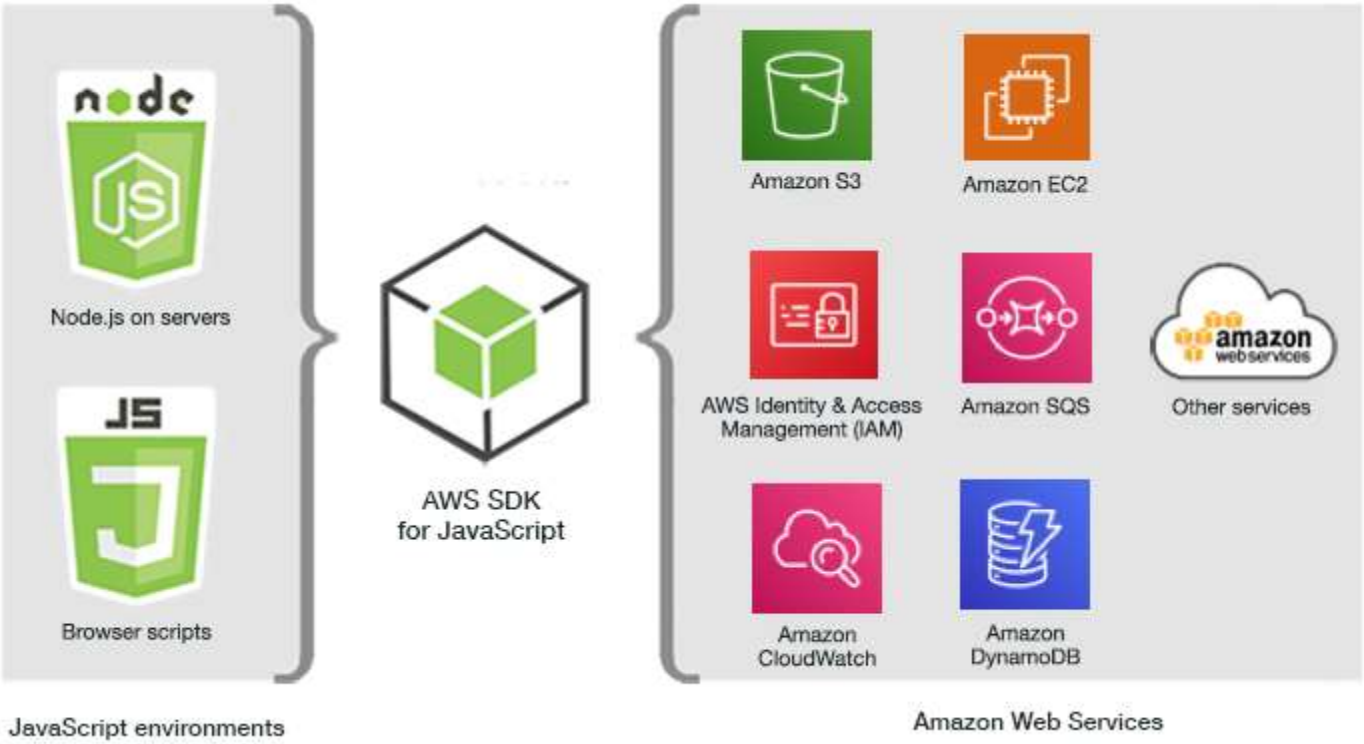
Node.js in Visual Studio Code

code.visualstudio.com/docs/nodejs/nodejs-tutorial

Node on Train (*scaffolding tool for full stack developer*)

nodeontrain.xyz

node.js: instrumente de dezvoltare



execuția aplicațiilor Node.js în „nori”: *AWS SDK for JS*

docs.aws.amazon.com/sdk-for-javascript/

aws.amazon.com/sdk-for-node-js/

(ツ)_/¯ ~/Dev/burkeholland/azure-casts/004

index.js | package.json

src ▸ index.js ▸ app.get("/") callback

```
7   app.get("/", function(req, res) {
8     res.send("Again I Go Unnoticed");
9   });
10
11  // this will throw an error
12  app.get("/read", function(req, res) {
13    fs.createReadStream("my-self-esteem.txt");
```

PROBLEMS OUTPUT TERMINAL ••• 1: zsh

```
[PM2] Starting /Users/burkeholland/Dev/burkeholland/azure-casts/004/src/index.js in fork_m
ode (1 instance)
[PM2] Done.
```

Name	id	mode	status	↺	cpu	memory
index	0	fork	online	0	0%	12.8 MB

```
➜ 004 git:(blog-post) ✗
```

blog-post* http://localhost:63343/ Ln 8, Col 29 Spaces: 2 UTF-8 LF JavaScript ESLint Prettier: ✓ 2

execuția aplicațiilor Node.js în „nori”: *Azure SDK for JS*

docs.microsoft.com/en-us/azure/javascript/

Azure Tools for Visual Studio Code

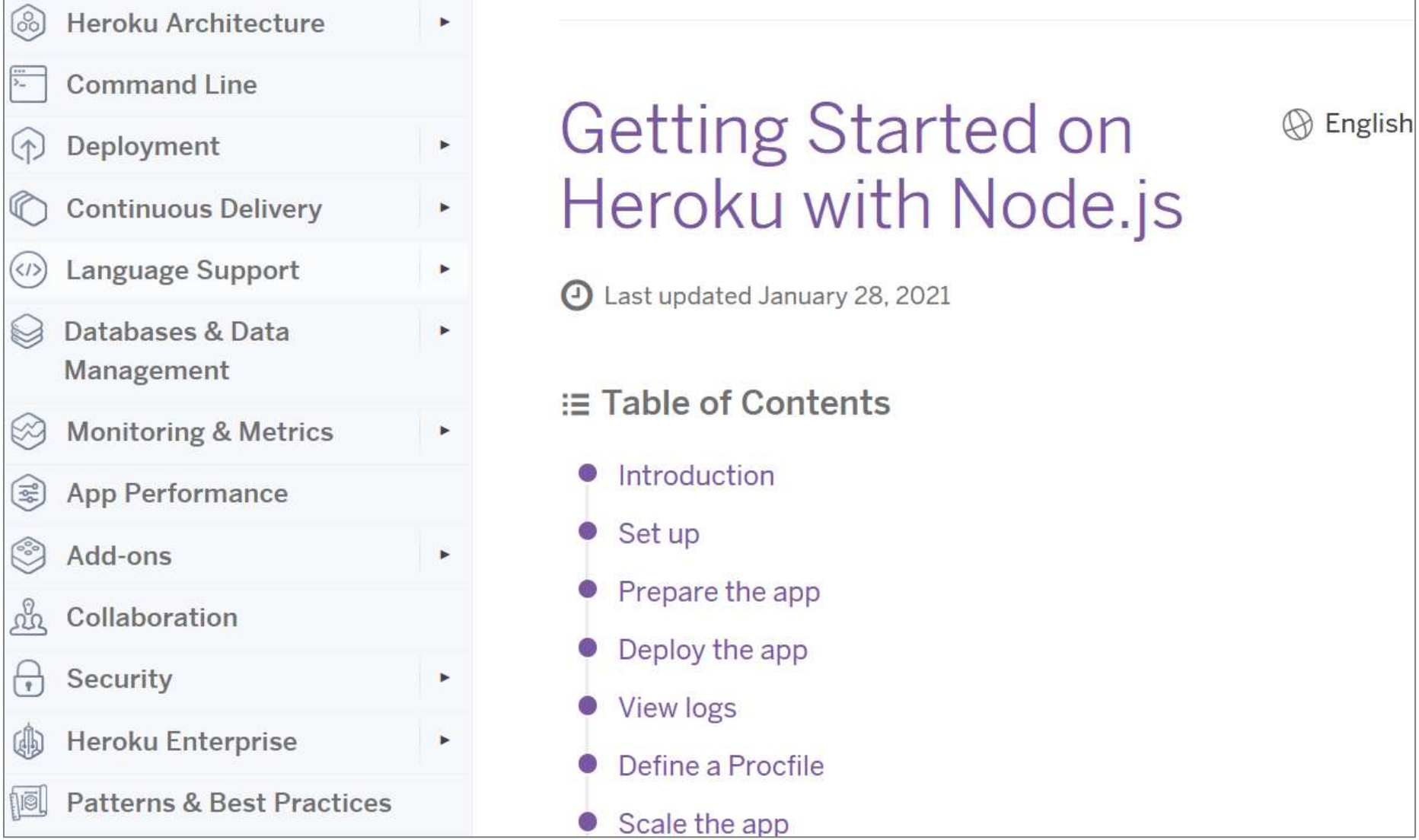
github.com/microsoft/vscode-node-azure-pack

```
// Import the Google Cloud client library
const {BigQuery} = require('@google-cloud/bigquery');

async function queryStackOverflow() {
  // Queries a public Stack Overflow dataset.

  // Create a client
  const bigqueryClient = new BigQuery();

  // The SQL query to run
  const sqlQuery = `SELECT
    CONCAT(
      'https://stackoverflow.com/questions/',
      CAST(id as STRING)) as url,
    view_count
    FROM \`bigquery-public-data.stackoverflow.posts_questions\`
    WHERE tags like '%google-bigquery%'
    ORDER BY view_count DESC
    LIMIT 10`;

  const options = {
    query: sqlQuery,
```

execuția aplicațiilor Node.js în „nori”:

Node.js on Google Cloud Platform – **cloud.google.com/nodejs/**

```
  // Run the query
  const [rows] = await bigqueryClient.query(options);
```



execuția aplicațiilor Node.js în „nori”: *Node.js on Heroku*

www.heroku.com/nodejs

Getting Started on Heroku with Node.js

devcenter.heroku.com/articles/getting-started-with-nodejs

node.js: instrumente de dezvoltare

În cazul DigitalOcean, a se explora tutorialele
www.digitalocean.com/community/tags/node-js/

node.js: instrumente de dezvoltare

În contextul generării statice de conținuturi oferite de API-uri (JAMstack), a se considera soluții precum Netlify în conjuncție cu generatoare de situri Web statice (*e.g.*, Docusaurus, Gatsby, Next.js, Nuxt) și sisteme de management al conținutului (Ghost, Strapi, Webiny,...)

jamstack.org/generators/

node.js: alternative

Deno

alternativă la Node.js inițiată în 2018 de Ryan Dahl, creatorul Node.js

a simple, modern and secure runtime for JavaScript and TypeScript that uses V8 and is built in Rust

deno.land/manual

node.js: alternative

Deno

nu utilizează npm (modulele se indică via URL-uri sau căi de directoare locale); nu există package.json

orice acțiune asincronă întoarce o promisiune (Promise)

pentru a accesa resurse externe (fișiere, rețea,...) trebuie specificate explicit permisiuni

erorile netratate conduc automat la „moartea” *script*-ului

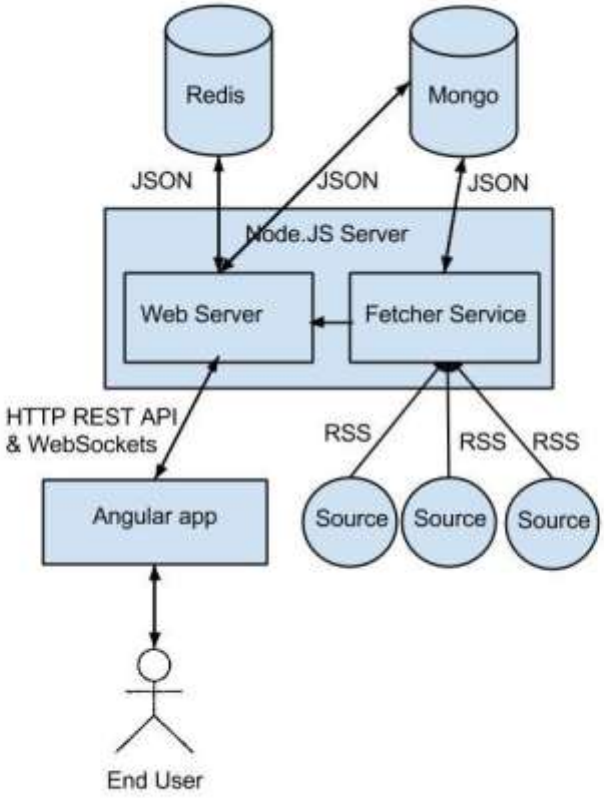
codul este automat plasat în *cache* la prima execuție (reîncărcarea se face manual)

Câteva studii de caz concrete…

node.js: exemplu – smartfetcher

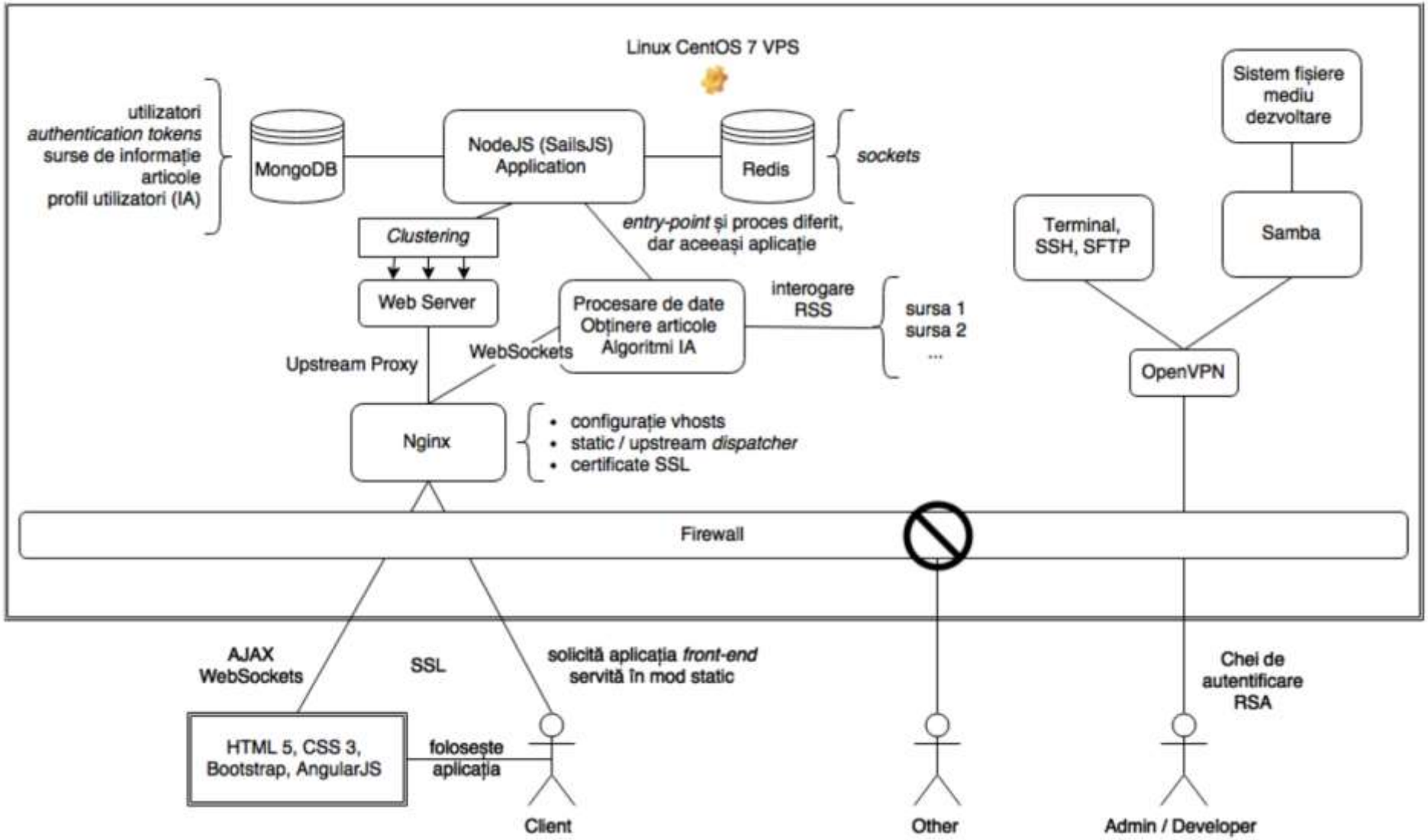
Scop: recomandare în timp-real de știri de interes (ziare, *blog*-uri,…), pe baza profilului utilizatorului și a interacțiunii sale cu sistemul

recurge la tehnici de învățare automată, în funcție și de sentimentul manifestat față de un articol dat



la nivel de server (*backend*): Node.js (*framework*-ul Sails)
API REST – eventual, folosit via WebSocket-uri
pentru comunicarea în timp-real cu clienții

pe partea client (*frontend*): Angular și Bootstrap



proiect de licență al absolvenților Raluca Jalaboi & Bogdan Spiridon (Facultatea de Informatică, UAIC Iași, 2015)

node.js: exemplu – muser

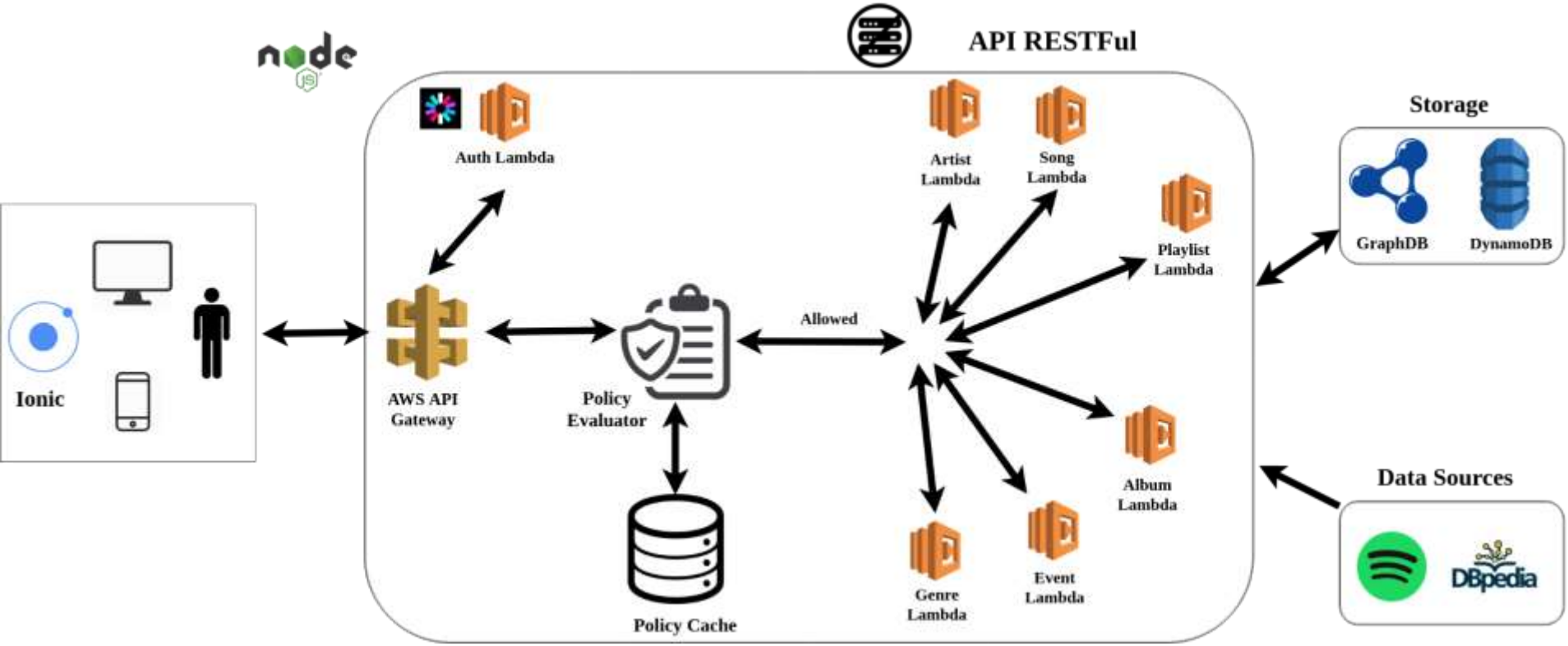
MuSeR (*Music Smart Enhancer*) – *a cross-platform semantic Web application developed for music lovers able to generate & manage in a "smart" way favorite playlists*

surse de date/cunoștințe externe: DBpedia + Spotify

implementare Node.js bazată pe micro-servicii Web: Serverless Framework – **serverless.com**

exploatare în „nori" via AWS (*Amazon Web Services*)

independența de client facilitată de Ionic

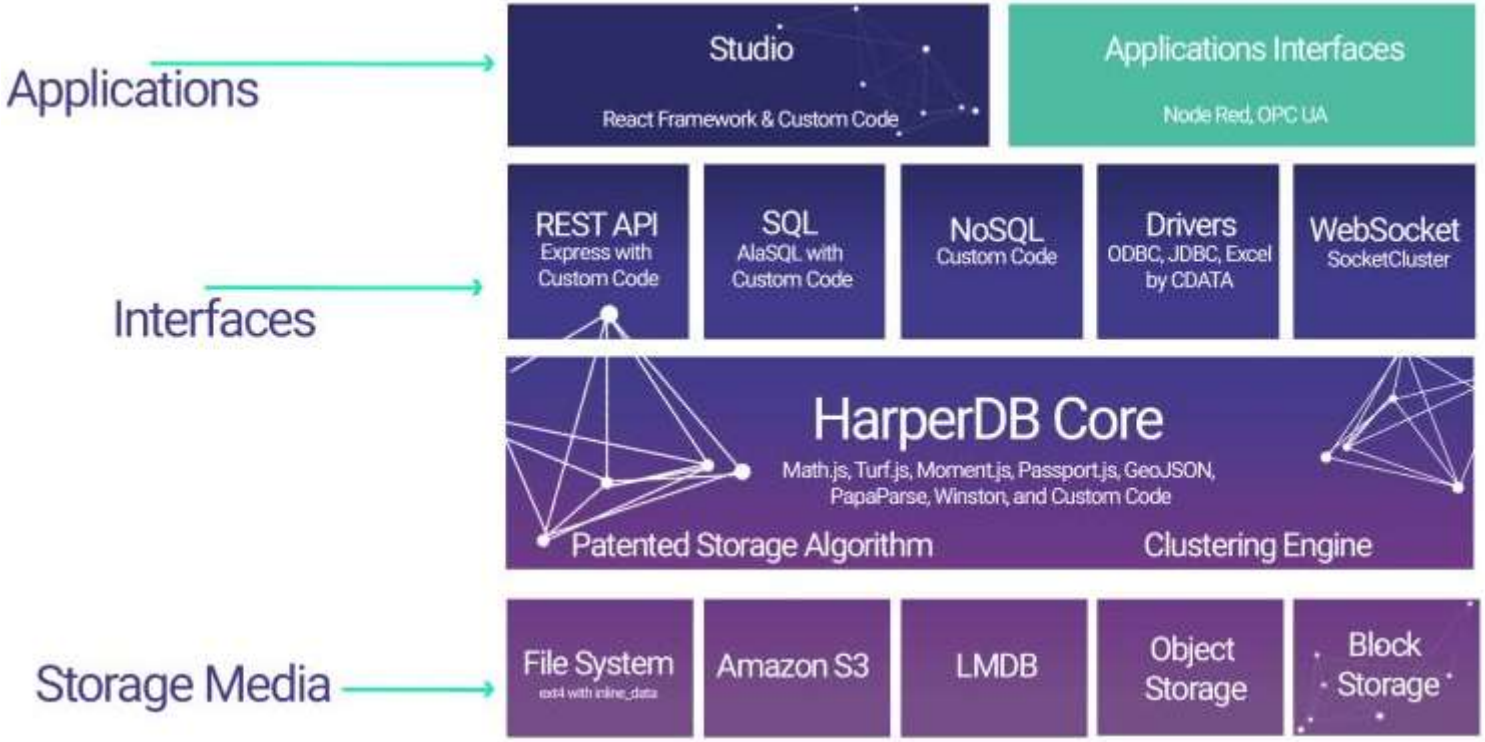


Lila-Ciprian Lazăr, Alexandru Cristea, Dan Cehan
(studii de master @ FII, UAIC Iași, 2017—2018)

cypmaster14.github.io/MuSer/

node.js: exemplu – harperdb

HarperDB – sistem scalabil conceput în Node.js pentru management de baze de date via un API REST: **harperdb.io**



hackernoon.com/brief-history-of-how-we-develop-harperdb-pb3z33qr

node.js: exemplu

Stive tehnologice (*tech stacks*) ce includ Node.js

tech stack
desemnează mulțimea tehnologiilor – limbaje, *framework*-uri, biblioteci, servere, soluții de interacțiune cu utilizatorul, utilitare, metodologii etc. – pe care un producător software le folosește pentru a dezvolta o aplicație (Web, mobilă,…)

What runs godbolt.org?

Web Server
- Nginx 1.14.0

Operating System
- Ubuntu

Build CI Systems
- webpack

Programming Language
- Node.js

CDN
- Amazon Cloudfront

Javascript Frameworks
- Express
- Hammer.js

Font
- Font Apple Sys

What runs dataplatform.cloud.ib..?

Analytics
- IBM Coremetrics
- MediaMath

Programming Language
- Node.js

Miscellaneous
- Segment
- TRUSTe Cookie Consent
- Lotame

Tag Managers
- Tealium

Javascript Frameworks
- Express
- React
- Socket.io

What runs unsplash.com?

Analytics
- Google Analytics UA
- comScore
- Scorecardresearch
- SpeedCurve

Cache
- Varnish

Build CI Systems
- webpack

Web Framework
- Cowboy

Programming Language
- Erlang
- Node.js

Javascript Frameworks
- Express
- React

Font
- Font Apple System

inspectarea tehnologiilor folosite de o aplicație Web cu instrumentul WhatRuns

Mixmax for Web
View Company Profile

Stack

Team

Application & Data (14)

jQuery
nginx
JavaScript
Node.js
React
MongoDB
Redis
Amazon S3
Sass
ExpressJS
Backbone.js
Electron
AWS Elastic Beanstalk
MongoDB Atlas

Utilities (12)

stackshare.io/mixmax/mixmax-for-web/

DevOps (10)

GitHub
npm
New Relic
Sublime Text
gulp
Babel
Sentry
Bower

ebay
View Company Profile

Stack | Team

Application & Data (9)

JavaScript | Node.js | Java | ES6 | Apache Tomcat | Cassandra | Hadoop | Oracle

Utilities (9)

stackshare.io/ebay/ebay/

DevOps (8)

Docker | Jenkins | Pingdom | BrowserStack | TeamCity | Apache Mesos | Honeybadger | AlertBot

Business Tools (7)

WordPress | G Suite | InVision | Balsamiq | Campaign Monitor | DocuSign | FogBugz

Food and More
View Company Profile
Stack
Team

Application & Data (14)

nginx
JavaScript
Node.js
Ubuntu
AngularJS
MongoDB
Amazon EC2
Amazon S3
ES6
Android SDK
Material Design for Angular
Flutter
Dart
LoopBack

Utilities (7)

CloudFlare
Postman
Google Maps
Amazon Route 53
Namecheap
Firebase Cloud Messaging
Conekta

DevOps (1

stackshare.io/food-and-more/food-and-more/

Business Tools (3)

Slack
G Suite
Whimsical Wireframes

BootstrapCDN
View Company Profile

Stack

Team

Application & Data (15)

jQuery
Bootstrap
JavaScript
Node.js
HTML5
Amazon S3
Ruby
Heroku
ExpressJS
Markdown
Pug
GNU Bash
Bootswatch
StackPath
CodeFund

Utilities (4)

stackshare.io/bootstrapcdn/bootstrapcdn/

DevOps (11)

GitHub
Git
Docker
npm
Visual Studio Code
Vim
Travis CI
ESLint
Mocha
Helm
Coveralls

node.js: alte resurse utile

Best Practices for Node.js Development
devcenter.heroku.com/articles/node-best-practices
github.com/goldbergyoni/nodebestpractices

Awesome Node.js
github.com/sindresorhus/awesome-nodejs

Node Weekly
nodeweekly.com

#nodejs Stories @ Hacker Noon
hackernoon.com/tagged/nodejs

Node.js Notes for Professionals
goalkicker.com/NodeJSBook/

rezumat

dezvoltarea de aplicații Web la nivel de server cu Node.js – caracteristici, module, exemple